嵌入向量

Embeddings

原文作者：@techNmak。

先列专业术语对照表，再提供逐页完整中文译文。每个原文页面的译文之后附原文矢量对照页，完整保留公式、图表、代码和英文引用，便于核对。

正文中文：LXGW WenKai（霞鹜文楷）；英文与数字：Times New Roman。原文对照页保留原始字体。

本文采用翻译辅助服务生成初稿，统一技术术语并检查结构、字体和导航；涉及精确数值、公式或实现细节时，可直接对照相邻原文页。

专业术语翻译对照表

英文原词	中文译名	缩写／符号
Embeddings	嵌入向量	
Token	词元	
Contrastive Learning	对比学习	
Euclidean Distance	欧氏距离	
Cosine Similarity	余弦相似度	
Representation	表示	
Normalization	归一化	
Hidden State	隐藏状态	
Tokenization	分词	
Transformer	Transformer 模型	Transformer
Dot Product	点积	
Vocabulary	词表	
Attention	注意力机制	Attention
Retrieval	检索	
Reranking	重排序	
Embedding	嵌入	
Chunking	分块	
Matrices	矩阵	
Softmax	Softmax 归一化	Softmax
Pooling	池化	
Queries	查询	
Vectors	向量	
Vector	向量	
Matrix	矩阵	
Models	模型	
Model	模型	
Mental Model	理解框架	
Training	训练	
Query	查询	Q
Tokens	词元	
Embedding Matrix	嵌入矩阵	
Tokenizer	分词器	

英文原词	中文译名	缩写／符号
Tensor	张量	
Matrix Multiplication	矩阵乘法	
Token Embeddings	词元嵌入	
Allocator Overhead	分配器开销	
Token Embedding	词元嵌入	
Precision	精度	
Weight Tying	权重绑定	
Gradient	梯度	
Static Embeddings	静态嵌入	
Word Embeddings	词嵌入	
Key	键	K
Pretraining	预训练	
Hidden Representation	隐藏表示	
Contextual Embeddings	上下文嵌入	
Contextual Embedding	上下文嵌入	
Feed-forward	前馈	
Unembedding	解嵌入	
Document Embedding	文档嵌入	
Activation	激活	
Scaling	缩放	
Output Projection	输出投影	
Word Embedding	词嵌入	
Serving	推理服务	
Sentence Embedding	句子嵌入	
Mean Pooling	平均池化	
Encoder	编码器	
Decoder	解码器	
Representation Space	表示空间	
Mask	掩码	
Sentence Embeddings	句子嵌入	
Generation	生成	
Benchmark	基准测试	
Baseline	基线	
Forward Pass	前向传播	

英文原词	中文译名	缩写／符号
Unit Vectors	单位向量	
Dropout	随机失活	
Value	值	V
Score Matrix	分数矩阵	
Retriever	检索器	
Index	索引	
Pipeline	流水线	
Quantization	量化	
Evidence	证据	
Dense Vectors	稠密向量	
Accuracy	准确性	
Embedding Space	嵌入空间	
Dynamic Range	动态范围	
Fine-tuning	微调	
Anisotropy	各向异性	
Dense Vector	稠密向量	
Values	值	
Core	核心	
Recall	召回率	
Document Embeddings	文档嵌入	
Generator	生成器	
Reranker	重排序器	

目录导航

点击章节标题跳转到中文译文，点击“原文”核对公式和图表。页码按本译本最终分页生成。

封面与说明 · 第 1 页

专业术语翻译对照表 · 第 2 页

嵌入向量（Embeddings） · 第 7 页 ［原文 1］

方向 · 第 9 页 ［原文 2］

1 - 嵌入（Embedding） 是什么 · 第 11 页 ［原文 3］

2 - 为何采用分布式表示 · 第 13 页 ［原文 4］

3 - 词元（Token） 嵌入矩阵（Embedding Matrix） · 第 15 页 ［原文 5］

4 - 查找是 矩阵（Matrix） 操作 · 第 17 页 ［原文 6］

5 - 参数计数和内存 · 第 19 页 ［原文 7］

6 - 如何学习 嵌入向量（Embeddings） · 第 21 页 ［原文 8］

7 - 静态 词嵌入（Word Embeddings） · 第 23 页 ［原文 9］

8 - 为什么静态字 向量（Vectors） 还不够 · 第 25 页 ［原文 10］

9 - 上下文嵌入（Contextual Embeddings） · 第 27 页 ［原文 11］

10 - 词元嵌入（Token Embeddings） VS 隐藏状态 · 第 29 页 ［原文 12］

11 - 位置和输入表示 · 第 31 页 ［原文 13］

12 - 输出 嵌入向量（Embeddings） 和 权重绑定（Weight Tying） · 第 33 页 ［原文 14］

13 - 从 词元（Tokens） 到一篇文本 向量（Vector） · 第 35 页 ［原文 15］

14 - 池化（Pooling） · 第 37 页 ［原文 16］

15 - 原始 LM 状态不会自动产生良好的语义 嵌入向量（Embeddings） · 第 39 页 ［原文 17］

16 - 暹罗猫和 BI-编码器（Encoder） 模型（Models） · 第 41 页 ［原文 18］

17 - 相似性：点积（Dot Product）、余弦和 L2 · 第 43 页 ［原文 19］

18 - 归一化（Normalization） · 第 45 页 ［原文 20］

19 - 工作几何形状示例 · 第 47 页 ［原文 21］

20 - 对比学习（Contrastive Learning） · 第 49 页 ［原文 22］

21 - 工作对比损失 · 第 51 页 ［原文 23］

22 - 负片和硬负片 · 第 53 页 ［原文 24］

23 - 密集 检索（Retrieval） · 第 55 页 ［原文 25］

24 - 查询（Query）/文档不对称和说明 · 第 57 页 ［原文 26］

25 - 嵌入（Embedding） 尺寸 · 第 59 页 ［原文 27］

26 - 俄罗斯套娃代表 · 第 61 页 ［原文 28］

27 - 各向异性（Anisotropy） 和 表示（Representation） 几何形状 · 第 63 页 ［原文 29］

28 - 单 向量（Vector）、多 向量（Vector） 和稀疏表示 · 第 65 页 ［原文 30］

29 - 多语言和多模式 嵌入向量（Embeddings） · 第 67 页 ［原文 31］

30 - 评估 嵌入向量（Embeddings） · 第 69 页 ［原文 32］

31 - 嵌入向量（Embeddings） 穿着破布 · 第 71 页 ［原文 33］
32 - 生产耦合和版本控制 · 第 73 页 ［原文 34］
33 - 透明的 Pytorch 实现 · 第 75 页 ［原文 35］
34 - 常见的实施错误 · 第 77 页 ［原文 36］
35 - 常见的误解 · 第 79 页 ［原文 37］
36 - 完整的 理解框架（Mental Model） · 第 81 页 ［原文 38］
参考 · 第 83 页 ［原文 39］

嵌入向量（Embeddings）

对应原文第 1 页；点击查看相邻原文对照页

理解 嵌入向量（Embeddings） 从 词元（Token） 查找表到现代语义表示 面向 AI 工程师、开发人员和认真学习者的简短技术手册 手册 08 - 理解 AI 系列

Understanding Embeddings

From Token Lookup Tables to Modern Semantic Representations

A short technical handbook for AI engineers, developers, and serious learners

@techNmak

Handbook 08 - Understanding AI Series

方向

对应原文第 2 页；点击查看相邻原文对照页

方向 本手册将阐明“嵌入（Embedding）”是现代人工智能中最重载的单词之一。它可以表示学习的 词元（Token） 表中的一行、Transformer 模型（Transformer） 内的上下文 隐藏状态（Hidden State）、合并句子向量（Vector）、用于 检索（Retrieval） 的文档 表示（Representation），甚至是跨文本和图像共享的 向量（Vector）。

本手册将这些含义分开并将它们联系起来。到最后，您应该能够解释：

- 为什么词元（Token） 查找表是学习参数矩阵（Matrix） 而不是定义数据库？

- 静态词向量（Vectors） 与上下文词元（Token） 状态有何不同；

- 可变长度文本如何通过 池化（Pooling） 或 模型（Model） 特定读出变为固定大小的 向量（Vector）；

- 为什么余弦相似度（Cosine Similarity）、点积（Dot Product）、归一化（Normalization） 和欧氏距离（Euclidean Distance） 不是可互换的选择；

- 对比学习（Contrastive Learning） 如何塑造空间，使有用的邻居变得紧密，而负面的变得可分离；

- 为什么检索（Retrieval） 模型（Models） 经常以不同方式对待查询（Queries） 和文档；

- 嵌入（Embedding） 维度发生什么变化，以及何时截断在数学上是合理的；

- 为什么向量（Vector） 空间只有与创建它的模型（Model）、预处理、目标和相似函数一起才有意义。

先决条件 对 向量（Vectors）、矩阵（Matrices）、点积、Softmax 归一化（Softmax） 和 Transformer 模型（Transformer） 隐藏状态的基本熟悉就足够了。早期关于 分词（Tokenization） 和 注意力机制（Attention） 的手册是有用的上下文，但本手册从第一原理重建了 嵌入（Embedding） 机制。

范围 这是一本关于学习表示的手册。它涉及密集的 检索（Retrieval）、RAG、多模态 模型（Models） 和 向量（Vector） 索引，只是为了解释 嵌入向量（Embeddings） 是如何生成和比较的。 HNSW、混合搜索、重排序（Reranking）、分块（Chunking） 和完整的 检索（Retrieval） 系统设计属于他们自己的手册。

研究锚点：[1]、[4]、[7]、[9]、[10]、[11]、[14]、[18] 理解 嵌入向量（Embeddings）

通篇使用的符号 V = 词表（Vocabulary） 尺寸； T = 序列长度； d_模型（Model） = Transformer 模型（Transformer） 隐藏/词元（Token）-嵌入（Embedding） 宽度； d = 通用 表示（Representation） 宽度； d_e = 最终文本 嵌入（Embedding） 尺寸（当它与 模型（Model） 隐藏宽度不同时）。节定义了它们出现的任何附加局部符号。

ORIENTATION

What this handbook will make clear

"Embedding" is one of the most overloaded words in modern AI. It can mean a row in a learned token table, a contextual hidden state inside a Transformer, a pooled sentence vector, a document representation used for retrieval, or even a vector shared across text and images.

This handbook separates those meanings and connects them. By the end, you should be able to explain:

- why a token lookup table is a learned parameter matrix rather than a database of definitions;
- how static word vectors differ from contextual token states;
- how variable-length text becomes a fixed-size vector through pooling or model-specific readout;
- why cosine similarity, dot product, normalization, and Euclidean distance are not interchangeable choices;
- how contrastive learning shapes a space so useful neighbors become close and negatives become separable;
- why retrieval models often treat queries and documents differently;
- what embedding dimension changes, and when truncation is mathematically justified;
- why a vector space is only meaningful together with the model, preprocessing, objective, and similarity function that created it.

Prerequisites

Basic comfort with vectors, matrices, dot products, softmax, and Transformer hidden states is enough. The earlier handbooks on tokenization and attention are useful context, but this handbook rebuilds the embedding mechanism from first principles.

Scope

This is a handbook about learned representations. It touches dense retrieval, RAG, multimodal models, and vector indexes only to explain how embeddings are produced and compared. HNSW, hybrid search, reranking, chunking, and full retrieval-system design belong in their own handbooks.

Notation used throughout

V = vocabulary size; T = sequence length; d_model = Transformer hidden/token-embedding width; d = a generic representation width; d_e = the final text-embedding dimension when it differs from the model hidden width. Sections define any additional local symbols where they appear.

Research anchors: [1], [4], [7], [9], [10], [11], [14], [18]

1 - 嵌入（Embedding） 是什么

对应原文第 3 页；点击查看相邻原文对照页

1 - 嵌入（Embedding） 是什么 向量（Vector） 空间的学习映射 嵌入（Embedding） 是将对象映射到有限维 向量（Vector） 的函数：

objectx 可能是 词元（Token） ID、单词、句子、文档、图像、代码片段或其他完全不同的东西。有用的部分不在于输出是“数字列表”。有用的部分是 训练（Training） 使 向量（Vector） 空间中的关系支持下游目标。

对于 词表（Vocabulary） 词元（Token），映射可能就像从 矩阵（Matrix） 中选择学习行一样简单。

对于一个句子，可能需要 Transformer 模型（Transformer） 加上 池化（Pooling） 或读出规则。对于 检索（Retrieval），训练（Training） 目标可以明确鼓励相关的 查询（Query） 文档对具有高相似性。

因此，嵌入（Embedding） 不具有独立于其 训练（Training） 进程的通用语义含义。

坐标417不会自动成为“否定”，坐标901也不会自动成为“主题”。 表示（Representation） 是分布式的：有用的信息通常跨多个维度和方向进行编码。

理解框架（Mental Model）。嵌入（Embedding）是一个学习的表示（Representation），而不是提前交给模型（Model）的语义坐标系。

研究锚点：[1]、[2]、[3]、[13] 理解 嵌入向量（Embeddings）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

1 - WHAT AN EMBEDDING IS

A learned map into a vector space

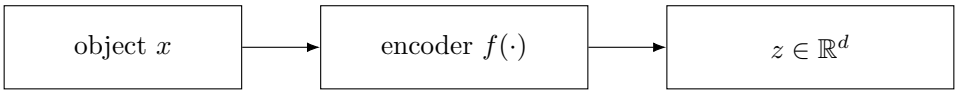
An embedding is a function that maps an object into a finite-dimensional vector:

$$f(x) \in \mathbb{R}^d.$$

The object x might be a token ID, a word, a sentence, a document, an image, a code snippet, or something else entirely. The useful part is not that the output is "a list of numbers." The useful part is that training makes relationships in the vector space support a downstream objective.

For a vocabulary token, the mapping may be as simple as selecting a learned row from a matrix. For a sentence, it may require a Transformer plus a pooling or readout rule. For retrieval, the training objective may explicitly encourage relevant query-document pairs to have high similarity.

An embedding therefore has no universal semantic meaning independent of its training process. Coordinate 417 is not automatically "negation" and coordinate 901 is not "topic." The representation is distributed: useful information is generally encoded across many dimensions and directions.



Mental model. An embedding is a learned representation, not a semantic coordinate system handed to the model in advance.

Research anchors: [1], [2], [3], [13]

2 - 为何采用分布式表示

对应原文第 4 页；点击查看相邻原文对照页

2 - 为什么采用分布式表示法 通过共享统计强度进行泛化 独热的 表示（Representation） 为每个 词表（Vocabulary） 项目提供了自己的独立轴。如果 词表（Vocabulary） 有 V 条目，则 tokeni 可以由 RV 中的向量（Vector） 表示，除了位置之外，该 向量（Vector） 处处为零

该编码唯一标识 词元（Token），但它不会使两个相关的 词元（Tokens） 接近。“cat”和“dog”就像“cat”和“compiler”一样正交。相反，学习密集的 表示（Representation） 在更小的空间 Rd 中为每个项目提供一个向量（Vector），在许多 模型（Models） 中具有 d≪V。

本吉奥等人。框架学习的连续单词表示作为对抗语言建模中维数灾难的一种方法：训练（Training） 示例可以影响对附近表示的预测，而不仅仅是精确的离散序列。后来的 word-向量（Vector） 方法（例如 word2vec 和 GloVe）使得大规模学习有用的几何结构变得更加便宜。 [1-3] 分布式这个词很重要。通常不会为概念分配一个专用维度。每个向量（Vector）使用多个坐标，每个坐标可以参与代表多个项目。

重要：密集并不自动意味着语义。随机密集表仍然只是随机表。几何形状变得有用，因为目标和数据在训练（Training） 期间塑造了它。

研究锚点：[1]、[2]、[3] 理解 嵌入向量（Embeddings）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

2 - WHY DISTRIBUTED REPRESENTATIONS

Generalize by sharing statistical strength

A one-hot representation gives each vocabulary item its own independent axis. If the vocabulary has V entries, token i can be represented by a vector in $\mathbb{R}^V$ that is zero everywhere except position i.

That encoding uniquely identifies the token, but it does not make two related tokens close. “cat” and “dog” are as orthogonal as “cat” and “compiler.” A learned dense representation instead gives each item a vector in a much smaller space $\mathbb{R}^d$, with $d \ll V$ in many models.

Bengio et al. framed learned continuous word representations as a way to fight the curse of dimensionality in language modeling: training examples can influence predictions for nearby representations rather than only exact discrete sequences. Later word-vector methods such as word2vec and GloVe made large-scale learning of useful geometric structure much cheaper. [1–3]

The word *distributed* matters. A concept is not normally assigned one dedicated dimension. Each vector uses many coordinates, and each coordinate can participate in representing many items.

Important. Dense does not automatically mean semantic. A random dense table is still just a random table. The geometry becomes useful because an objective and data shape it during training.

Research anchors: [1], [2], [3]

3 - 词元（Token） 嵌入矩阵（Embedding Matrix）

对应原文第 5 页；点击查看相邻原文对照页

3 - 词元（Token） 嵌入矩阵（Embedding Matrix） LLM 看到的第一个学习的 表示（Representation） 让 分词器（Tokenizer） 在 {0,...,V - 1} 中生成 词元（Token） ID。标准 词元（Token）-嵌入（Embedding） 层存储 矩阵（Matrix）

RowEi 是与 词元（Token） IDi 关联的学习到的 向量（Vector）。

对于 词元（Token） ID 序列

嵌入（Embedding） 查找返回

在批处理中，实现通常返回具有逻辑形状的 张量（Tensor）

PyTorch 将 nn.嵌入（Embedding） 直接描述为一个查找表，其中存储 嵌入向量（Embeddings） 的固定字典和 嵌入（Embedding） 的大小。 [19]

词元（Token） ID

嵌入（Embedding） 行是 模型（Model） 参数。它不是 分词器（Tokenizer） 对 词元（Token） 的定义，并且 词元（Token） ID 本身不包含几何图形。 ID 仅选择行。

理解框架（Mental Model）。 分词（Tokenization） 选择离散符号。 嵌入矩阵（Embedding Matrix） 将这些符号转换为学习的连续 向量（Vectors）。

研究锚点：[4]、[7]、[19] 理解 嵌入向量（Embeddings）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

3 - THE TOKEN EMBEDDING MATRIX

The first learned representation an LLM sees

Let a tokenizer produce token IDs in $\{0, \ldots, V-1\}$. A standard token-embedding layer stores a matrix

$$E \in \mathbb{R}^{V \times d_{\text{model}}}.$$

Row E_i is the learned vector associated with token ID i.

For a sequence of token IDs

$$(t_1, t_2, \ldots, t_n),$$

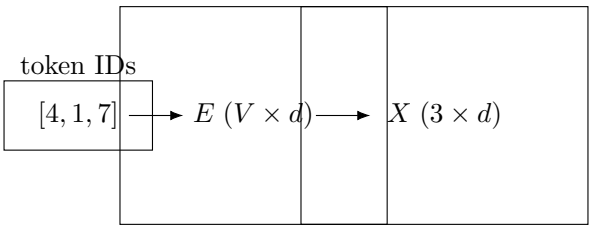
the embedding lookup returns

$$X = [E_{t_1}; E_{t_2}; \ldots; E_{t_n}] \in \mathbb{R}^{n \times d_{\text{model}}}.$$

In a batch, an implementation commonly returns a tensor with logical shape

$$B \times T \times d_{\text{model}}.$$

PyTorch describes `nn.Embedding` directly as a lookup table that stores embeddings for a fixed dictionary and embedding size. [19]



The embedding row is a model parameter. It is not the tokenizer's definition of the token, and token IDs themselves do not contain geometry. IDs merely select rows.

Mental model. Tokenization chooses discrete symbols. The embedding matrix turns those symbols into learned continuous vectors.

Research anchors: [4], [7], [19]

4 - 查找是 矩阵（Matrix） 操作

对应原文第 6 页；点击查看相邻原文对照页

4 - LOOKUP 是一个 矩阵（Matrix） 操作 单热乘法和行选择是等价的 嵌入（Embedding） 查找通常被实现为索引，因为它是高效的，但也可以理解为普通的 矩阵乘法（Matrix Multiplication）。

假设

包含四个维度三的 词元嵌入（Token Embeddings）。 词元（Token） ID2 对应于 one-hot 行 向量（Vector）

然后

这正是基于零的索引下 E 的第 2 行。

对于一个序列，将 one-hot 行堆叠到 O∈ RT×V 中。然后

真正的实现通常不会构造巨大的稀疏独热 矩阵（Matrix）。它仅收集请求的行。

这种等价性很有用，因为它消除了“嵌入（Embedding） 层”的一些神秘感。它是从离散基础到低维连续空间的学习线性映射，通过索引进行有效评估。

重要：查找不会平均、搜索或检索最近的 词元（Token）。给定一个 ID，该层会准确选择相应的参数行。

研究锚点：[1]、[19] 理解 嵌入向量（Embeddings）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

4 - LOOKUP IS A MATRIX OPERATION

One-hot multiplication and row selection are equivalent

Embedding lookup is often implemented as indexing because it is efficient, but it can be understood as ordinary matrix multiplication.

Suppose

$$E = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 2 & 1 \\ 3 & 1 & 0 \\ 2 & 2 & 2 \end{bmatrix}$$

contains four token embeddings of dimension three. Token ID 2 corresponds to the one-hot row vector

$$e_2^\top = [0, 0, 1, 0].$$

Then

$$e_2^\top E = [3, 1, 0],$$

which is exactly row 2 of E under zero-based indexing.

For a sequence, stack one-hot rows into $O \in \mathbb{R}^{T \times V}$. Then

$$X = OE \in \mathbb{R}^{T \times d}.$$

A real implementation does not usually construct the enormous sparse one-hot matrix. It gathers only the requested rows.

This equivalence is useful because it removes some mystery from an "embedding layer." It is a learned linear map from a discrete basis into a lower-dimensional continuous space, evaluated efficiently by indexing.

Important. Lookup does not average, search, or retrieve the nearest token. Given an ID, the layer selects the corresponding parameter row exactly.

Research anchors: [1], [19]

5 - 参数计数和内存

对应原文第 7 页；点击查看相邻原文对照页

5 - 参数计数和内存 词表（Vocabulary） 大小乘以 嵌入（Embedding） 宽度 词元嵌入（Token Embedding） 表包含 V dmodel 参数。这可能是较小语言 模型（Model） 的一个重要部分，特别是当 词表（Vocabulary） 很大时。

举一个说明性的配置：

然后

嵌入（Embedding） 参数。对于每个参数两个字节，原始参数有效负载为 268,435,456 字节 = 256 MiB。

如果 模型（Model） 还使用相同形状的单独未绑定输出 矩阵（Matrix），则在此示例中，矩阵（Matrix） 将需要另外 256 MiB 的两字节参数存储。

此计算仅是参数有效负载。 训练（Training） 内存还可以包括梯度、优化器状态、更高的 精度（Precision） 主权重、分片元数据和 分配器开销（Allocator Overhead）。

因此，词表（Vocabulary） 设计与 模型（Model） 尺寸相互作用。添加 词元（Tokens） 并不是免费的：每个新的 词表（Vocabulary） 行都会将模型学习参数添加到未绑定的输入表中。

重要： 嵌入（Embedding）尺寸和词表（Vocabulary）尺寸是不同的轴。增加 V 会增加行。

增加 d 会使每行更宽，如果 词元嵌入（Token Embedding） 直接输入 模型（Model），还会更改隐藏宽度。

研究锚点：[4]、[5]、[19] 理解 嵌入向量（Embeddings）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

5 - PARAMETER COUNT AND MEMORY

Vocabulary size times embedding width

The token embedding table contains

$$V\, d_{\text{model}}$$

parameters. This can be a nontrivial fraction of a smaller language model, especially when the vocabulary is large.

Take an illustrative configuration:

$$V = 32{,}768, \qquad d_{\text{model}} = 4096.$$

Then

$$Vd = 134{,}217{,}728$$

embedding parameters. At two bytes per parameter, the raw parameter payload is

$$268{,}435{,}456 \text{ bytes} = 256 \text{ MiB}.$$

If a model also used a separate untied output matrix of the same shape, that matrix would require another 256 MiB of two-byte parameter storage in this example.

This calculation is only the parameter payload. Training memory can additionally include gradients, optimizer states, higher-precision master weights, sharding metadata, and allocator overhead.

Vocabulary design therefore interacts with model size. Adding tokens is not free: every new vocabulary row adds d_{model} learned parameters to an untied input table.

Important. Embedding dimension and vocabulary size are different axes. Increasing V adds rows. Increasing d makes every row wider and also changes the hidden width if the token embedding feeds the model directly.

Research anchors: [4], [5], [19]

6 - 如何学习 嵌入向量（Embeddings）

对应原文第 8 页；点击查看相邻原文对照页

6 - 如何学习 嵌入向量（Embeddings） 行会移动，因为下游损失将梯度发回 嵌入（Embedding） 表不需要特殊的“语义”损失即可变得有用。如果它参与可微的 模型（Model），则任务丢失可以通过反向传播更新所选行。

假设 词元（Token） IDt 选择 rowEt。网络的其余部分计算lossL。反向传播提供

然后优化器会更新该行以及 模型（Model） 的其他参数。当前批次中未使用的 词元（Tokens） 行可能不会在简单的 嵌入（Embedding） 层中接收直接查找 梯度（Gradient），尽管 权重绑定（Weight Tying） 或 矩阵（Matrix） 的其他用途可以创建其他 梯度（Gradient） 路径。

在许多示例中，参与类似预测上下文的 词元（Tokens） 可以被推向支持类似下游计算的表示。但并不要求欧几里得邻域成为人类可解释的，除非目标使几何变得有用。

早期神经语言 模型（Models） 显式学习分布式单词 向量（Vectors） 和概率 模型（Model）。现代变形金刚在更大的规模上做了同样的事情：词元嵌入（Token Embeddings） 与网络的其余部分进行端到端学习。[1,4]

理解框架（Mental Model）。 嵌入（Embedding） 表不是粘贴到 模型（Model） 上的预训练知识。在端到端 模型（Model） 中，它是由相同的 训练（Training） 信号塑造的众多参数中的一组。

研究锚点：[1]、[4]、[5] 理解 嵌入向量（Embeddings）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

6 - HOW EMBEDDINGS ARE LEARNED

Rows move because downstream loss sends gradients back

An embedding table does not need a special "semantic" loss to become useful. If it participates in a differentiable model, the task loss can update the selected rows through backpropagation.

Suppose token ID t selects row E_t. The rest of the network computes a loss $\mathcal{L}$. Backpropagation provides

$$\frac{\partial \mathcal{L}}{\partial E_t}.$$

An optimizer then updates that row along with the model's other parameters. Rows for tokens not used in the current batch may receive no direct lookup gradient in a simple embedding layer, although weight tying or other uses of the matrix can create additional gradient paths.

Over many examples, tokens that participate in similar predictive contexts can be pushed toward representations that support similar downstream computations. But there is no requirement that Euclidean neighborhoods become human-interpretable unless the objective makes that geometry useful.

Early neural language models explicitly learned distributed word vectors jointly with the probability model. Modern Transformers do the same at much larger scale: token embeddings are learned end-to-end with the rest of the network. [1,4]

Mental model. The embedding table is not pretrained knowledge glued onto the model. In an end-to-end model, it is one set of parameters among many, shaped by the same training signal.

Research anchors: [1], [4], [5]

7 - 静态 词嵌入（Word Embeddings）

对应原文第 9 页；点击查看相邻原文对照页

7 - 静态 词嵌入（Word Embeddings） Word2vec 和 GloVe 使 向量（Vector） 几何变得实用 在上下文 Transformer 之前，常见的 表示（Representation） 为每种单词类型分配一个 向量（Vector）。

Word2vec 引入了高效的预测架构，用于从本地上下文中学习连续单词 向量（Vectors），包括 CBOW 和 skip-gram。相反，GloVe 在词与词共现统计上训练了全局对数双线性回归 模型（Model）。 [2,3]

细节有所不同，但两个家族都帮助建立了一个持久的想法：文本中的分布规律可以在学习的 向量（Vector） 空间中编码为有用的几何图形。

静态嵌入（Static Embeddings） 使最近邻搜索、相似性、聚类和特征转移等操作变得廉价。他们还揭示了 向量（Vectors） 之间差异的惊人规律，尽管类比行为不应被误认为是意义的通用代数。

键（Key） 限制位于 wordstatic 中。无论句子涉及金钱还是河流，诸如“银行”之类的类型都接收相同的基数 向量（Vector）。上下文只能稍后由另一个 模型（Model） 合并。

银行 电子银行 河岸银行相同类型 向量（Vector）

重要。一些单词-向量（Vector） 空间的著名线性规律是学习表示的经验属性，并不能保证每个语义关系都是干净的 向量（Vector） 偏移。

研究锚点：[2]、[3] 理解 嵌入向量（Embeddings）

7 - STATIC WORD EMBEDDINGS

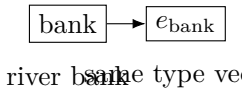
Word2vec and GloVe made vector geometry practical

Before contextual Transformers, a common representation assigned one vector to each word type. Word2vec introduced efficient predictive architectures for learning continuous word vectors from local contexts, including CBOW and skip-gram. GloVe instead trained a global log-bilinear regression model on word-word co-occurrence statistics. [2,3]

The details differ, but both families helped establish a durable idea: distributional regularities in text can be encoded as useful geometry in a learned vector space.

Static embeddings made operations such as nearest-neighbor search, similarity, clustering, and feature transfer inexpensive. They also exposed striking regularities in differences between vectors, although analogy behavior should not be mistaken for a universal algebra of meaning.

The key limitation is in the word *static*. A type such as “bank” receives the same base vector whether the sentence concerns money or a river. Context can only be incorporated later by another model.



Important. The famous linear regularities of some word-vector spaces are empirical properties of learned representations, not a guarantee that every semantic relation is a clean vector offset.

Research anchors: [2], [3]

8 - 为什么静态字 向量（Vectors） 还不够

对应原文第 10 页；点击查看相邻原文对照页

8 - 为什么静态单词 向量（Vectors） 不够 含义随上下文而变化 自然语言充满了多义性和上下文依赖性。每个字类型一个 向量（Vector） 必须将该类型的每次使用压缩为一个 表示（Representation）。

上下文 表示（Representation） 模型（Models） 更改了 表示（Representation） 的单位。 ELMo 使用双向语言 模型（Model） 状态生成单词表示作为整个输入句子的函数。

然后，BERT 通过在 预训练（Pretraining） 期间在左右上下文上调节每个 词元（Token） 来学习深度双向 Transformer 模型（Transformer） 表示。 [6,7]

现在“bank”出现两次

- “她在银行存了现金。”

- “他们坐在河岸上。”可以从相同的 词元嵌入（Token Embedding） 行开始，但在 模型（Model） 处理其周围环境后以不同的上下文隐藏状态结束。

这种区别是现代 嵌入（Embedding） 术语的基础：

静态 词元（Token）/类型 嵌入（Embedding） 与离散 词表（Vocabulary） 项关联的学习 向量（Vector）。

上下文 词元（Token） 表示（Representation） 上下文计算后为 词元（Token） 出现生成的 隐藏状态（Hidden State）。

理解框架（Mental Model）。 词元（Token） 表提供起始 表示（Representation）。 Transformer 模型（Transformer） 将该起点转变为上下文相关状态。

研究锚点：[6]、[7] 理解 嵌入向量（Embeddings）

8 - WHY STATIC WORD VECTORS ARE NOT ENOUGH

Meaning changes with context

Natural language is full of polysemy and context dependence. A single vector per word type must compress every use of that type into one representation.

Contextual representation models changed the unit of representation. ELMo generated word representations as functions of the entire input sentence using bidirectional language-model states. BERT then learned deep bidirectional Transformer representations by conditioning each token on both left and right context during pretraining. [6,7]

Now the two occurrences of “bank” in

- “She deposited cash at the bank.”
- “They sat on the river bank.”

can begin from the same token embedding row yet end with different contextual hidden states after the model processes their surroundings.

That distinction is foundational for modern embedding terminology:

Static token/type embedding	A learned vector associated with a discrete vocabulary item.
Contextual token representation	A hidden state produced for a token occurrence after contextual computation.

Mental model. The token table supplies a starting representation. The Transformer turns that starting point into a context-dependent state.

Research anchors: [6], [7]

9 - 上下文嵌入（Contextual Embeddings）

对应原文第 11 页；点击查看相邻原文对照页

9 - 上下文嵌入（Contextual Embeddings） 相同的词元（Token） ID可以产生不同的隐藏向量（Vectors）让tokenti有base embeddingeti。 Transformer 模型（Transformer） 层堆栈计算上下文状态

尽管相同的 词元（Token） ID 无论出现在何处都会选择相同的基本 rowet，但 hi 取决于周围的 词元（Tokens）、位置处理、注意力机制（Attention）、前馈（Feed-forward） 计算、归一化（Normalization）以及正在检查的图层。

因此，如果不指定具体指哪个 表示（Representation），则说“词元（Token） X 的 嵌入（Embedding）”是不准确的。可能的含义包括：

- 输入查找向量Et；

- 隐藏状态（Hidden State） 在第ℓ层出现一次；

- 源自许多词元（Token） 状态的汇集文本嵌入（Embedding）；

- 用于对词表（Vocabulary） 项目进行评分的输出/解嵌入（Unembedding） 行。

上下文表示也是特定于层的。最终的 隐藏状态（Hidden State） 不能与中间层互换，因为两者具有相同的维度。

相同的基行Et上下文A→h(A)

上下文 B→h(B)

重要。“上下文嵌入（Contextual Embedding）”是通用语言，但对象通常是上下文生成器生成的 隐藏表示（Hidden Representation），而不是存储在输入 嵌入（Embedding） 表中的行。

研究锚点：[6]、[7] 理解 嵌入向量（Embeddings）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

9 - CONTEXTUAL EMBEDDINGS

The same token ID can produce different hidden vectors

Let token t_i have base embedding e_{t_i}. A Transformer layer stack computes contextual states

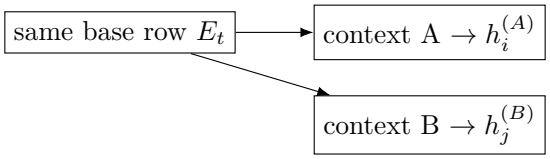
$$(h_1, \ldots, h_T) = F(e_{t_1}, \ldots, e_{t_T}; \theta).$$

Although the same token ID selects the same base row e_t wherever it appears, h_i depends on surrounding tokens, position treatment, attention, feed-forward computation, normalization, and the layer being inspected.

It is therefore imprecise to say "the embedding of token X" without specifying which representation is meant. Possible meanings include:

- the input lookup vector E_t;
- the hidden state for one occurrence at layer ℓ;
- a pooled text embedding derived from many token states;
- an output/unembedding row used to score vocabulary items.

Contextual representations are also layer-specific. The final hidden state is not interchangeable with an intermediate layer simply because both have the same dimensionality.



Important. "Contextual embedding" is common language, but the object is usually a hidden representation produced by a contextualizer, not a row stored in the input embedding table.

Research anchors: [6], [7]

10 - 词元嵌入（Token Embeddings） VS 隐藏状态

对应原文第 12 页；点击查看相邻原文对照页

10 - 词元嵌入（Token Embeddings） VS 隐藏状态 不要折叠三种不同的 向量（Vector） 类型 现代语言 模型（Models） 公开了多个具有相同宽度的 向量（Vectors），这使得它们很容易混淆。

令牌 嵌入（Embedding） 上下文状态 文本 嵌入（Embedding） 输入 词元（Token） ID 序列状态文本 / 词元（Token） 状态 上下文相关?

否 是 通常是 每个词汇项一个 词元（Token） 出现句子/文档/等。

典型作用 模型（Model） 输入内部计算相似度/检索（Retrieval）/任务特征 第一个是参数查找。第二个是激活（Activation）。第三个是故意暴露的 表示（Representation），用于比较较大的单元，例如句子或文档。

将所有三个称为“嵌入向量（Embeddings）”不一定是错误的，但在推理内存、训练（Training）、相似性或重用时，它会变得危险。 词表（Vocabulary） 嵌入矩阵（Embedding Matrix） 存在于 模型（Model） 参数中。上下文状态是根据输入生成的。 文档嵌入（Document Embedding） 通常可以预先计算并存储在模型（Model） 之外。

理解框架（Mental Model）。总是问：嵌入的是什么，通过哪个功能，在哪一层或读出，以及出于什么目的?

研究锚点：[4]、[7]、[9]、[10] 理解 嵌入向量（Embeddings）

10 - TOKEN EMBEDDINGS VS HIDDEN STATES

Do not collapse three different vector types

Modern language models expose several vectors with the same width, which makes them easy to confuse.

	Token embedding	**Contextual state**	**Text embedding**
Input	token ID	sequence states	text / token states
Context dependent?	no	yes	usually yes
One per	vocab item	token occurrence	sentence/document/etc.
Typical role	model input	internal computation	similarity/retrieval/task feature

The first is a parameter lookup. The second is an activation. The third is a representation deliberately exposed for comparing larger units such as sentences or documents.

Calling all three "embeddings" is not necessarily wrong, but it becomes dangerous when reasoning about memory, training, similarity, or reuse. A vocabulary embedding matrix lives in the model parameters. Contextual states are produced per input. A document embedding can often be precomputed and stored outside the model.

Mental model. Always ask: *what is being embedded, by which function, at which layer or readout, and for what objective?*

Research anchors: [4], [7], [9], [10]

11 - 位置和输入表示

对应原文第 13 页；点击查看相邻原文对照页

11 - 位置和输入表示 词元（Token） 身份只是一种成分 除了 词元（Token） 身份之外，Transformer 模型（Transformer） 还需要某种方式来表示序列顺序。原始的 Transformer 模型（Transformer） 在第一层之前向 词元嵌入（Token Embeddings） 添加了位置编码。 BERT 使用学习到的 词元（Token）、段和位置 嵌入向量（Embeddings）。其他现代 模型（Models） 在 注意力机制（Attention） 计算中使用 RoPE，而不是向输入添加位置 向量（Vector）。 [4,7]

因此，一般输入 表示（Representation） 不应硬编码为 词元嵌入（Token Embedding）+位置 嵌入（Embedding）

持久的区别是：

- 词元嵌入（Token Embedding）：代表离散的词表（Vocabulary） 项；

- 位置机制：为网络提供有关顺序或相对位置的信息；

- 情境化：通过网络混合序列中的信息。

在原始的 Transformer 模型（Transformer） 中，在添加位置编码之前，词元嵌入（Token Embeddings） 乘以√dmodel。 缩放（Scaling） 是特定于体系结构的设计选择，而不是 嵌入（Embedding） 查找的通用定义的一部分。 [4]

重要。不要说 RoPE“改变了 词元嵌入（Token Embedding）”。标准RoPE作用于注意力机制（Attention）内部的Q/K；它是围绕上下文计算的位置机制，而不是替代的 词表（Vocabulary） 表。

研究锚点：[4]、[7] 理解 嵌入向量（Embeddings）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

11 - POSITION AND INPUT REPRESENTATIONS

Token identity is only one ingredient

A Transformer needs some way to represent sequence order in addition to token identity. The original Transformer added positional encodings to token embeddings before the first layer. BERT used learned token, segment, and position embeddings. Other modern models use RoPE inside the attention computation instead of adding a position vector to the input. [4,7]

So a general input representation should not be hard-coded as

$$\text{token embedding} + \text{position embedding}$$

for every architecture.

The durable distinction is:

- **token embedding**: represents the discrete vocabulary item;
- **position mechanism**: gives the network information about order or relative position;
- **contextualization**: mixes information across the sequence through the network.

In the original Transformer, token embeddings are multiplied by $\sqrt{d_{\text{model}}}$ before positional encodings are added. That scaling is an architecture-specific design choice, not part of the universal definition of embedding lookup. [4]

Important. Do not say RoPE "changes the token embedding." Standard RoPE acts on Q/K inside attention; it is a positional mechanism surrounding the contextual computation, not a replacement vocabulary table.

Research anchors: [4], [7]

12 - 输出 嵌入向量（Embeddings） 和 权重绑定（Weight Tying）

对应原文第 14 页；点击查看相邻原文对照页

12 - 输出 嵌入向量（Embeddings） 和 权重绑定（Weight Tying） 词表（Vocabulary） 矩阵（Matrix） 可以出现在 模型（Model） 的两侧 语言 模型（Model） 必须最终将 隐藏状态（Hidden State） 转换为 词表（Vocabulary） 项上的 logits。使用 row-向量（Vector） 约定，可以编写未绑定的 输出投影（Output Projection）

Press 和 Wolf 将输出 矩阵（Matrix） 本身分析为 词嵌入（Word Embedding），并建议将输入和输出 嵌入向量（Embeddings） 绑定在语言 模型（Models） 中。 [5]

如果输入表为E∈RV×d并且权重绑定，则logits可以表示为

（取决于特定于架构的偏差或 缩放（Scaling））。

权重绑定（Weight Tying） 减少了参数计数，因为 模型（Model） 不再存储具有相同 词表（Vocabulary） 按宽度形状的两个独立的 矩阵（Matrices）。它还耦合了他们的学习：相同的参数行参与表示输入符号和评分输出符号。

Transformer 模型（Transformer） 论文共享了目标输入 嵌入（Embedding） 和预 Softmax 归一化（Softmax） 线性变换权重，并在其翻译设置中另外共享了源/目标 嵌入向量（Embeddings）。 [4]

重要。“嵌入矩阵（Embedding Matrix）”可以指输入查找表、被视为 嵌入（Embedding） 的输出 词表（Vocabulary） 投影，或绑定的 矩阵（Matrix） 推理服务（Serving） 两个角色。说明你指的是哪一个。

研究锚点：[4]、[5] 理解 嵌入向量（Embeddings）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

12 - OUTPUT EMBEDDINGS AND WEIGHT TYING

The vocabulary matrix can appear on both sides of the model

A language model must eventually convert a hidden state into logits over vocabulary items. With row-vector convention, an untied output projection can be written

$$\ell = hW_{\text{out}}, \qquad W_{\text{out}} \in \mathbb{R}^{d \times V}.$$

Press and Wolf analyzed the output matrix itself as a word embedding and recommended tying input and output embeddings in language models. [5]

If the input table is $E \in \mathbb{R}^{V \times d}$ and weights are tied, the logits can be expressed as

$$\ell = hE^{\top}$$

(up to architecture-specific biases or scaling).

Weight tying reduces parameter count because the model no longer stores two independent matrices with the same vocabulary-by-width shape. It also couples their learning: the same parameter rows participate in both representing input symbols and scoring output symbols.

The Transformer paper shared the target input embedding and pre-softmax linear transformation weights, and additionally shared source/target embeddings in its translation setup. [4]

Important. "Embedding matrix" can refer to the input lookup table, the output vocabulary projection viewed as an embedding, or a tied matrix serving both roles. State which one you mean.

Research anchors: [4], [5]

13 - 从 词元（Tokens） 到一篇文本 向量（Vector）

对应原文第 15 页；点击查看相邻原文对照页

13 - 从 词元（Tokens） 到一篇文本 向量（Vector） 可变长度序列输入，固定大小 表示（Representation） 输出 检索（Retrieval），相似性系统通常每个句子、段落或文档需要一个 向量（Vector）。然而，Transformer 模型（Transformer） 自然会为每个 词元（Token） 生成一个 隐藏状态（Hidden State）：

因此，文本-嵌入（Embedding） 模型（Model） 需要一个读出规则

常见的选择包括 平均池化（Mean Pooling） 超过有效的 词元（Tokens）、指定的分类 词元（Token）、最终的非填充 词元（Token）、学习的 池化（Pooling） 头或 模型（Model） 特定的投影。

Sentence-BERT 明确地将 池化（Pooling） 操作添加到 BERT/RoBERTa 输出，并研究平均值、最大值和 CLS 池化（Pooling）。其在原论文中的默认值是平均池化（Mean Pooling）。 [9]

基于 解码器（Decoder） 的现代 嵌入（Embedding） 模型（Models） 可以使用不同的读数。 Qwen3 嵌入（Embedding） 附加一个 EOS 词元（Token） 并将 EOS 处的最后一层 隐藏状态（Hidden State） 作为文本 表示（Representation）。 [18]

不存在通用的“正确的 池化（Pooling）”。读数是经过训练的 模型（Model） 定义的一部分。

理解框架（Mental Model）。 句子嵌入（Sentence Embedding） 不仅仅是“BERT 向量（Vector）”或“LLM 向量（Vector）”。它是 编码器（Encoder） 的输出加上特定的读出规则，通常针对预期的相似性任务进行训练或验证。

研究锚点：[9]、[14]、[18]、[20] 理解 嵌入向量（Embeddings）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

13 - FROM TOKENS TO ONE TEXT VECTOR

Variable-length sequence in, fixed-size representation out

Retrieval and similarity systems usually need one vector per sentence, passage, or document. A Transformer, however, naturally produces one hidden state per token:

$$H = [h_1; \ldots; h_T] \in \mathbb{R}^{T \times d}.$$

A text-embedding model therefore needs a readout rule

$$z = g(H, \text{mask}) \in \mathbb{R}^{d_e}.$$

Common choices include mean pooling over valid tokens, a designated classification token, the final non-padding token, a learned pooling head, or a model-specific projection.

Sentence-BERT explicitly adds a pooling operation to BERT/RoBERTa outputs and studied mean, max, and CLS pooling. Its default in the original paper was mean pooling. [9]

Modern decoder-based embedding models can use a different readout. Qwen3 Embedding appends an EOS token and takes the final-layer hidden state at EOS as the text representation. [18]

There is no universal "correct pooling." The readout is part of the trained model definition.

Mental model. A sentence embedding is not simply "the BERT vector" or "the LLM vector." It is the output of an encoder plus a specific readout rule, usually trained or validated for the intended similarity task.

Research anchors: [9], [14], [18], [20]

14 - 池化（Pooling）

对应原文第 16 页；点击查看相邻原文对照页

14 - 池化（Pooling） 掩码（Mask） 填充并保留 模型（Model） 的预期读数 对于具有隐藏 stateshi 和二进制 maskmi∈{ 0, 1} 的 平均池化（Mean Pooling），

我=1mihi

掩码（Mask） 很重要，因为填充 词元（Tokens） 不应默默地将 向量（Vector） 拉向填充 表示（Representation）。

考虑三个有效的 词元（Token） 状态和一个填充位置：

蒙面 平均池化（Mean Pooling） 给出

包含填充会产生 [3, 3]，一个完全不同的 向量（Vector）。

池化（Pooling） 也可以是 模型（Model） 特定的。 Sentence Transformers 目前支持的策略包括 CLS、max、mean、加权平均值和 last-词元（Token） 池化（Pooling）。 [20]

重要。 池化（Pooling） 不是一个无害的后处理选择。如果 模型（Model） 使用特定的读数进行训练，则更改它可能会使您进入 表示空间（Representation Space），而 训练（Training） 目标从未优化。

研究锚点：[9]、[14]、[18]、[20] 理解 嵌入向量（Embeddings）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

14 - POOLING

Mask padding and preserve the model's intended readout

For mean pooling with hidden states h_i and a binary mask $m_i \in \{0, 1\}$,

$$z = \frac{\sum_{i=1}^{T} m_i h_i}{\sum_{i=1}^{T} m_i}.$$

The mask matters because padding tokens should not silently pull the vector toward a padding representation.

Consider three valid token states and one padded position:

$$h_1 = [1, 0],\ h_2 = [0, 2],\ h_3 = [2, 1],\ h_4 = [9, 9], \quad m = [1, 1, 1, 0].$$

Masked mean pooling gives

$$z = \frac{[1, 0] + [0, 2] + [2, 1]}{3} = [1, 1].$$

Including padding would instead produce $[3, 3]$, a completely different vector.

Pooling can also be model-specific. Sentence Transformers currently supports strategies including CLS, max, mean, weighted mean, and last-token pooling. [20]

Important. Pooling is not a harmless post-processing choice. If a model was trained with a specific readout, changing it can move you into a representation space the training objective never optimized.

Research anchors: [9], [14], [18], [20]

15 - 原始 LM 状态不会自动产生良好的语义 嵌入向量（Embeddings）

对应原文第 17 页；点击查看相邻原文对照页

15 - RAW W LM 状态不会自动产生良好的语义 嵌入向量（Embeddings） 生成（Generation） 目标和相似性目标不同 预训练语言 模型（Model） 可以为任何文本生成隐藏状态，但这并不能保证简单的池化 向量（Vector） 具有用于语义相似性的有用余弦几何。

Sentence-BERT 在其 2019 年论文中通过 BERT 模型（Models） 和评估设置清晰地展示了这一点：

与专门针对 句子嵌入（Sentence Embeddings） 进行微调的 编码器（Encoder） 相比，直接平均 BERT 输出或使用 CLS 输出在语义文本相似性方面表现不佳。重点不是旧的 基准测试（Benchmark） 号码。关键是客观不匹配。 [9]

生成式或掩码语言 模型（Model） 目标训练隐藏状态以支持 词元（Token） 预测和 模型（Model） 的内部计算。 检索（Retrieval） 嵌入（Embedding） 目标反而关心成对几何：在选定的相似性规则下，相关文本应该比不相关文本得分更高。

因此，基于现代LLM的嵌入（Embedding） 模型（Models）针对嵌入（Embedding）任务微调基础模型（Models），而不是假设基础模型（Model）的隐藏状态已经形成理想的检索（Retrieval）空间。例如，Qwen3 嵌入（Embedding） 使用对比目标和定义的读数来训练基础 模型（Model） 骨干网。 [18]

重要。“取最后一个 隐藏状态（Hidden State） 并对其进行平均”是 基线（Baseline），而不是高质量 嵌入向量（Embeddings） 的通用配方。

研究锚点：[9]、[11]、[18]、[21] 理解 嵌入向量（Embeddings）

15 - RAW LM STATES ARE NOT AUTOMATICALLY GOOD SEMANTIC EMBEDDINGS

Generation objectives and similarity objectives are different

A pretrained language model can produce hidden states for any text, but that does not guarantee that a simple pooled vector has useful cosine geometry for semantic similarity.

Sentence-BERT showed this sharply for the BERT models and evaluation setup in its 2019 paper: directly averaging BERT outputs or using the CLS output performed poorly on semantic textual similarity compared with an encoder fine-tuned specifically for sentence embeddings. The point is not the old benchmark number. The point is the objective mismatch. [9]

A generative or masked-language-model objective trains hidden states to support token prediction and the model's internal computation. A retrieval embedding objective instead cares about pairwise geometry: relevant texts should score higher than irrelevant texts under a chosen similarity rule.

Modern LLM-based embedding models therefore fine-tune foundation models for embedding tasks rather than assuming the base model's hidden states already form an ideal retrieval space. Qwen3 Embedding, for example, trains the foundation-model backbone with a contrastive objective and a defined readout. [18]

Important. "Take the last hidden state and average it" is a baseline, not a universal recipe for high-quality embeddings.

Research anchors: [9], [11], [18], [21]

16 - 暹罗猫和 BI-编码器（Encoder） 模型（Models）

对应原文第 18 页；点击查看相邻原文对照页

16 - 暹罗和 BI-编码器（Encoder） 模型（Models） 独立编码项目，比较它们便宜 bi-编码器（Encoder） 将两个输入独立映射到 向量（Vectors）：

然后计算可分解的相似度，例如

编码器可以共享参数或者是分开的。 Sentence-BERT 使用连体/三元组风格的结构来使独立的句子表示具有可比性。 DPR 使用不同的问题和段落编码器，两者都基于原始作品中的 BERT。 [9,10]

关键的系统属性是可分解性。可以在不知道未来 查询（Query） 的情况下预先计算文档 向量（Vectors）。在 查询（Query） 时间，只有 查询（Query） 在 向量（Vector） 搜索之前需要新的 前向传播（Forward Pass）。

跨 编码器（Encoder） 是不同的：它联合处理该对，这可以使 模型（Model） 更丰富的 词元（Token） 级交互，但会阻止对每个可能的 查询（Query） 的文档分数进行独立预计算。

查询（Query） fq zq 文档 fd zd 相似度

理解框架（Mental Model）。 嵌入（Embedding） 模型（Models） 用一些成对交互的丰富性来换取可重用的表示和可扩展的比较。

研究锚点：[9]、[10]、[14] 理解 嵌入向量（Embeddings）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

16 - SIAMESE AND BI-ENCODER MODELS

Encode items independently, compare them cheaply

A bi-encoder maps two inputs independently into vectors:

$$z_q = f_q(q), \qquad z_d = f_d(d),$$

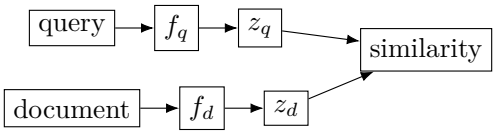
then computes a decomposable similarity such as

$$s(q, d) = z_q^\top z_d.$$

The encoders may share parameters or be separate. Sentence-BERT uses siamese/triplet-style structures to make independent sentence representations comparable. DPR uses distinct question and passage encoders, both based on BERT in the original work. [9,10]

The crucial systems property is decomposability. Document vectors can be precomputed without knowing the future query. At query time, only the query needs a fresh forward pass before vector search.

A cross-encoder is different: it processes the pair jointly, which can model richer token-level interactions but prevents independent precomputation of document scores for every possible query.



Mental model. Embedding models trade some pairwise interaction richness for reusable representations and scalable comparison.

Research anchors: [9], [10], [14]

17 - 相似性：点积（Dot Product）、余弦和 L2

对应原文第 19 页；点击查看相邻原文对照页

17 - 相似性：点积（Dot Product）、余弦和 L2 该指标是 模型（Model） 合约的一部分 三种常见的比较是

点积（Dot Product） 取决于方向和 向量（Vector） 大小。余弦去除幅度并比较角度。 欧氏距离（Euclidean Distance） 测量原始比例中的几何间距。

如果 向量（Vectors） 已 L2 归一化，使得 ∥u∥ = ∥v∥ = 1，则

和

因此，对于 单位向量（Unit Vectors），最大化 cosine/点积（Dot Product） 和最小化 欧氏距离（Euclidean Distance） 会产生相同的排名。

DPR 故意使用 点积（Dot Product）。 SimCSE 的标准物镜使用 余弦相似度（Cosine Similarity）。 E5 使用按温度缩放的余弦。这些是 训练（Training） 选择，而不仅仅是可互换的 查询（Query） 时间开关。[10,11,14]

重要。最安全的产生式规则是使用 嵌入（Embedding） 模型（Model） 指定的相似函数和 归一化（Normalization） 约定。即使 向量（Vector） 尺寸不变，更改它们也会改变排名。

研究锚点：[10]、[11]、[14]、[20] 理解 嵌入向量（Embeddings）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

17 - SIMILARITY: DOT PRODUCT, COSINE, AND L2

The metric is part of the model contract

Three common comparisons are

$$\text{dot}(u, v) = u^\top v,$$

$$\cos(u, v) = \frac{u^\top v}{\|u\|_2 \|v\|_2},$$

$$d_2(u, v) = \|u - v\|_2.$$

Dot product depends on both direction and vector magnitude. Cosine removes magnitude and compares angle. Euclidean distance measures geometric separation in the original scale.

If vectors are L2-normalized so that $\|u\| = \|v\| = 1$, then

$$u^\top v = \cos(u, v)$$

and

$$\|u - v\|_2^2 = 2 - 2u^\top v.$$

Therefore, for unit vectors, maximizing cosine/dot product and minimizing Euclidean distance induce the same ranking.

DPR deliberately used dot product. SimCSE's standard objective uses cosine similarity. E5 uses cosine scaled by temperature. These are training choices, not merely interchangeable query-time switches. [10,11,14]

Important. The safest production rule is to use the similarity function and normalization convention specified by the embedding model. Changing them can change rankings even if the vector dimensions are unchanged.

Research anchors: [10], [11], [14], [20]

18 - 归一化（Normalization）

对应原文第 20 页；点击查看相邻原文对照页

18 - 归一化（Normalization） 单位长度改变乐谱可以编码的 L2 归一化（Normalization） 映射

在 归一化（Normalization） 之后，每个非零 向量（Vector） 都位于单位超球面上。大小不再有助于点积相似性。

当 训练（Training） 物镜是有角度的时，这可能是理想的。如果 模型（Model） 被训练为使用 向量（Vector） 范数作为分数的一部分，也可能是错误的。

Sentence Transformers 明确指出标准化 嵌入向量（Embeddings） 上的 点积（Dot Product） 等效于 余弦相似度（Cosine Similarity），同时避免在比较时冗余重新 归一化（Normalization）。 [20]

归一化（Normalization） 还与维度截断相互作用。 Matryoshka 表示（Representation） Learning 指出，当使用归一化表示时，应针对每个嵌套维度独立处理 归一化（Normalization），而不是截断归一化的完整向量（Vector） 并假设前缀保持单位长度。 [15]

因此，一个实际的不变量是：

编码约定 + 归一化（Normalization） 约定 + 公制约定必须一致。

重要：不要仅标准化 查询（Queries） 或仅标准化文档，除非 模型（Model） 明确定义了非对称评分规则。

研究锚点：[11]、[15]、[20] 理解 嵌入向量（Embeddings）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

18 - NORMALIZATION

Unit length changes what the score can encode

L2 normalization maps

$$z \mapsto \hat{z} = \frac{z}{\|z\|_2}.$$

After normalization, every nonzero vector lies on the unit hypersphere. Magnitude no longer contributes to dot-product similarity.

This can be desirable when the training objective is angular. It can also be wrong if the model was trained to use vector norm as part of the score.

Sentence Transformers explicitly notes that dot product on normalized embeddings is equivalent to cosine similarity, while avoiding redundant re-normalization at comparison time. [20]

Normalization also interacts with dimensional truncation. Matryoshka Representation Learning notes that when normalized representations are used, normalization should be handled independently for each nested dimension rather than truncating a normalized full vector and assuming the prefix remains unit length. [15]

A practical invariant is therefore:

encode convention + normalization convention + metric convention must agree.

Important. Do not normalize only queries or only documents unless the model explicitly defines that asymmetric scoring rule.

Research anchors: [11], [15], [20]

19 - 工作几何形状示例

对应原文第 21 页；点击查看相邻原文对照页

19 - 工作几何示例 相同方向，不同大小

他们的规范是

点积是

余弦相似度为 cos(q,d 1) = 50

余弦(q,d 2) = 24

欧几里得距离是

因此 d1 与 q 完全对齐，但在原始欧几里得空间中距离更远，因为它的大小加倍了。 点积（Dot Product）强烈支持它，因为幅度也有贡献。

将所有向量（Vectors）归一化后，余弦和点积（Dot Product）一致，并且欧氏距离（Euclidean Distance）成为相同角度的单调变换。

理解框架（Mental Model）。在指定 表示（Representation） 缩放（Scaling） 和度量之前，“最近”是未定义的。

研究锚点：[10]、[20]理解嵌入向量（Embeddings）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

19 - WORKED GEOMETRY EXAMPLE

Same direction, different magnitude

Take

$$q = [3, 4], \qquad d_1 = [6, 8], \qquad d_2 = [4, 3].$$

Their norms are

$$\|q\| = 5, \quad \|d_1\| = 10, \quad \|d_2\| = 5.$$

Dot products are

$$q^\top d_1 = 50, \qquad q^\top d_2 = 24.$$

Cosine similarities are

$$\cos(q, d_1) = \frac{50}{5 \cdot 10} = 1,$$

$$\cos(q, d_2) = \frac{24}{5 \cdot 5} = 0.96.$$

The Euclidean distances are

$$\|q - d_1\| = 5, \qquad \|q - d_2\| = \sqrt{2} \approx 1.414.$$

So d_1 is perfectly aligned with q but farther away in raw Euclidean space because its magnitude is doubled. Dot product strongly favors it because magnitude also contributes.

After normalizing all vectors, cosine and dot product agree, and Euclidean distance becomes a monotonic transformation of the same angle.

Mental model. "Nearest" is undefined until you specify the representation scaling and the metric.

Research anchors: [10], [20]

20 - 对比学习（Contrastive Learning）

对应原文第 22 页；点击查看相邻原文对照页

20 - 对比学习（Contrastive Learning） 用积极和消极的方式塑造空间 现代文本 嵌入（Embedding） 模型（Models） 经常以对比目标进行训练。直觉很简单：增加相关对的相似性，并减少相对于竞争负片的相似性。

锚点 xi、正 x+ i、批量大小 N、相似性和温度的通用批量目标

ℓi =- log exp(s(zi,z +

j=1 exp(s(zi,z +

SimCSE 使用此形式与 余弦相似度（Cosine Similarity） 和批内否定。其无监督版本使用独立的 随机失活（Dropout） 掩码生成同一句子的两种表示；它的监督版本使用蕴涵对作为正例，使用矛盾示例作为硬负例。 [11]

DPR 使用类似的 Softmax 归一化（Softmax） 对正面段落和负面段落进行排名损失。 E5 类似地训练带有对比学习（Contrastive Learning） 的文本对和批内否定。 [10,14]

温度控制分数差异对 Softmax 归一化（Softmax） 的影响程度。较小的 τ 会放大相似性的差异，但有用的值（Value） 取决于 模型（Model） 和 训练（Training）。

重要。 对比学习（Contrastive Learning） 并没有教导“语义相似性”的抽象普遍概念。它根据 训练（Training） 数据中的正/负对、任务、采样策略和相似函数来教授几何。

研究锚点：[10]、[11]、[13]、[14]、[18] 理解 嵌入向量（Embeddings）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

20 - CONTRASTIVE LEARNING

Shape the space with positives and negatives

Modern text embedding models are often trained with a contrastive objective. The intuition is simple: increase similarity for related pairs and reduce it relative to competing negatives.

A common in-batch objective for anchor x_i, positive x_i^+, batch size N, similarity s, and temperature τ is

$$\ell_i = -\log \frac{\exp(s(z_i, z_i^+)/\tau)}{\sum_{j=1}^{N} \exp(s(z_i, z_j^+)/\tau)}.$$

SimCSE uses this form with cosine similarity and in-batch negatives. Its unsupervised version produces two representations of the same sentence using independent dropout masks; its supervised version uses entailment pairs as positives and contradiction examples as hard negatives. [11]

DPR uses an analogous softmax ranking loss over a positive passage and negatives. E5 similarly trains text pairs with contrastive learning and in-batch negatives. [10,14]

Temperature controls how sharply score differences affect the softmax. A smaller τ magnifies differences in similarity, but the useful value is model- and training-dependent.

Important. Contrastive learning does not teach an abstract universal notion of "semantic similarity." It teaches geometry according to the positive/negative pairs, tasks, sampling strategy, and similarity function in the training data.

Research anchors: [10], [11], [13], [14], [18]

21 - 工作对比损失

对应原文第 23 页；点击查看相邻原文对照页

21 - 有效对比损失 一个正数与两个负数竞争假设一个锚点有 余弦相似度（Cosine Similarity）

具有积极意义和相似之处

有两个否定。设τ = 0.5。

指数分数是

分配给正数的 Softmax 归一化（Softmax） 概率为

损失是

如果正相似度上升而负相似度保持不变，则 p+ 增加且损失下降。如果硬负例变得与锚点更加相似，它会收到更多的概率质量，并且损失会增加。

这个玩具计算说明了为什么消极选择很重要。 梯度（Gradient） 不仅由正对决定，还由分母中存在的替代对决定。

重要：绝对相似性分数在任意 嵌入（Embedding） 模型（Models） 之间不能直接比较。温度、归一化（Normalization）、模型（Model） 训练（Training） 和分数分布都很重要。

研究锚点：[10]、[11]、[18] 理解 嵌入向量（Embeddings） 与批内阴性的联系在批内设置中，这个三项分母与下一页 B x B 查询（Query） 文档 分数矩阵（Score Matrix） 的一行的想法相同：一个指定的阳性与用作阴性的其他批次项目竞争。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

21 - WORKED CONTRASTIVE LOSS

One positive competes against two negatives

Suppose an anchor has cosine similarity

$$s^+ = 0.8$$

with its positive and similarities

$$s_1^- = 0.3, \qquad s_2^- = 0.0$$

with two negatives. Let $\tau = 0.5$.

The exponentiated scores are

$$e^{0.8/0.5} \approx 4.9530, \quad e^{0.3/0.5} \approx 1.8221, \quad e^{0/0.5} = 1.$$

The softmax probability assigned to the positive is

$$p^+ = \frac{4.9530}{4.9530 + 1.8221 + 1} \approx 0.6370.$$

The loss is

$$\ell = -\log p^+ \approx 0.4509.$$

If the positive similarity rises while negatives stay fixed, p^+ increases and the loss falls. If a hard negative becomes more similar to the anchor, it receives more probability mass and the loss rises.

This toy calculation shows why negative choice matters. The gradient is shaped not just by the positive pair but by the alternatives present in the denominator.

Important. Absolute similarity scores are not directly comparable across arbitrary embedding models. Temperature, normalization, model training, and score distributions all matter.

Connection to in-batch negatives

In the in-batch setting, this three-item denominator is the same idea as one row of the B x B query-document score matrix on the next page: one designated positive competes against the other batch items used as negatives.

Research anchors: [10], [11], [18]

22 - 负片和硬负片

对应原文第 24 页；点击查看相邻原文对照页

22 - 负数和硬负数 分母教导边界 在 检索（Retrieval） 训练（Training） 中，大多数可能的文档与给定的查询（Query） 无关。面临的挑战是选择能够教导有用的区别而不引入太多假阴性的否定。

DPR 比较了随机阴性、BM25 检索的阴性以及与其他问题配对的段落。它还展示了批内负片如何重用批次中的其他段落，从 B 查询（Query） 文档对生成 B×B 分数矩阵（Score Matrix）。 [10]

对于

矩阵（Matrix）

包含批次中的每个 查询（Query） 文档 点积（Dot Product）。对角线对可以是正数，而非对角线对如果确实不相关，则可以作为负数。

硬底片特别有价值，因为它们在较弱的 检索器（Retriever） 下看起来似乎合理，或者与 查询（Query） 共享表面特征。但实际相关的“负面”会造成矛盾的监督。

现代目标越来越多地包括假阴性过滤或更丰富的阴性池。例如，当 Qwen3 嵌入（Embedding） 的基于相似性的标准表明某些候选者可能表现得像假阴性时，就会屏蔽某些候选者。 [18]

理解框架（Mental Model）。对比 训练（Training） 的质量不仅取决于正面例子的结构，还取决于负面例子的结构。

研究锚点：[10]、[11]、[14]、[18] 理解 嵌入向量（Embeddings）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

22 - NEGATIVES AND HARD NEGATIVES

The denominator teaches the boundary

In retrieval training, most possible documents are irrelevant to a given query. The challenge is choosing negatives that teach useful distinctions without introducing too many false negatives.

DPR compared random negatives, BM25-retrieved negatives, and passages paired with other questions. It also showed how in-batch negatives reuse the other passages in a batch, producing a $B \times B$ score matrix from B query-document pairs. [10]

For

$$Q \in \mathbb{R}^{B \times d}, \qquad D \in \mathbb{R}^{B \times d},$$

the matrix

$$S = QD^{\top}$$

contains every query-document dot product in the batch. Diagonal pairs can be positives, while off-diagonal pairs serve as negatives if they are truly irrelevant.

Hard negatives are especially valuable because they look plausible under a weaker retriever or share surface features with the query. But an actually relevant "negative" creates contradictory supervision.

Modern objectives increasingly include false-negative filtering or richer negative pools. Qwen3 Embedding, for example, masks certain candidates when its similarity-based criterion suggests they may behave like false negatives. [18]

Mental model. Contrastive training quality depends as much on the structure of negatives as on the positive examples.

Research anchors: [10], [11], [14], [18]

23 - 密集 检索（Retrieval）

对应原文第 25 页；点击查看相邻原文对照页

23 - 密集检索 预计算文档，编码 查询（Query），搜索 向量（Vector） 空间 密集 检索器（Retriever） 为每个文档或段落构建一个 嵌入（Embedding），将这些 向量（Vectors） 存储在 索引（Index） 中，然后嵌入 查询（Query） 并搜索高分向量（Vectors）。

DPR 形式化了经典的双 编码器（Encoder） 模式：

昂贵的段落 编码器（Encoder） 在语料库中离线运行。在线查询（Query）的费用是一个查询（Query）编码加上向量（Vector）搜索。 [10]

这就是为什么 嵌入（Embedding） 质量和搜索系统质量是不同的问题。对不良 嵌入向量（Embeddings）的完美精确最近邻搜索仍然会根据错误的几何形状返回邻居。相反，强大的 嵌入（Embedding） 模型（Model） 可能会因错误的相似性度量、陈旧的 向量（Vectors）、糟糕的 分块（Chunking） 或过于激进的近似搜索而受到损害。

密集的 检索（Retrieval） 也不会使词汇 检索（Retrieval） 过时。 E5 的分析明确指出了 BM25 保留优势的情况，特别是在精确词汇匹配和某些长尾或长文档任务方面。 [14]

重要。 嵌入（Embedding） 模型（Model） 定义评分空间。 向量（Vector） 索引（Index） 是一种用于搜索该空间的算法。不要将 表示（Representation） 学习与 索引（Index） 结构混淆。

研究锚点：[10]、[14]理解嵌入向量（Embeddings）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

23 - DENSE RETRIEVAL

Precompute documents, encode the query, search the vector space

A dense retriever builds an embedding for each document or passage, stores those vectors in an index, then embeds a query and searches for high-scoring vectors.

DPR formalized the classic dual-encoder pattern:

$$E_Q(q) \in \mathbb{R}^d, \qquad E_P(p) \in \mathbb{R}^d,$$

$$\text{sim}(q, p) = E_Q(q)^\top E_P(p).$$

The expensive passage encoder runs offline for the corpus. Online query cost is one query encoding plus vector search. [10]

This is why embedding quality and search-system quality are separate problems. A perfect exact nearest-neighbor search over bad embeddings still returns neighbors according to the wrong geometry. Conversely, a strong embedding model can be undermined by the wrong similarity metric, stale vectors, bad chunking, or overly aggressive approximate search.

Dense retrieval also does not make lexical retrieval obsolete. E5's analysis explicitly noted cases where BM25 retained advantages, especially for exact lexical matching and certain long-tail or long-document tasks. [14]

Important. An embedding model defines a scoring space. A vector index is an algorithm for searching that space. Do not confuse representation learning with index structure.

Research anchors: [10], [14]

24 - 查询（Query）/文档不对称和说明

对应原文第 26 页；点击查看相邻原文对照页

24 - 查询（Query）/文档不对称和说明 两侧并不总是表示同一事物 检索（Retrieval） 查询（Query） 和文档扮演不同的语义角色。许多 嵌入（Embedding） 模型（Models） 都明确表明了这种不对称性。

E5 使用一个共享的 编码器（Encoder），但在前面添加不同的文本标识符：“查询（Query）:”表示 查询（Queries），“passage:”表示段落。该论文报告说，以这种方式打破对称性对于某些 检索（Retrieval） 任务非常重要。 [14]

较新的指令感知 嵌入（Embedding） 模型（Models） 概括了这个想法。 Qwen3 嵌入（Embedding） 将任务指令与 查询（Query） 连接起来，同时保持文档不变，从而允许以请求的任务为条件确定相关性。 [18]

这意味着生产 流水线（Pipeline） 必须保留 模型（Model） 的输入协议。使用 查询（Query） 模板对语料库进行编码、省略所需的前缀或在索引后更改指令可能会导致不匹配，即使每个 向量（Vector） 都具有预期的维度。

对于对称任务（例如通用句子相似性），双方可以使用相同的文本角色。对于非对称 检索（Retrieval）、查询（Query）/文档约定可以是学习几何的一部分。

理解框架（Mental Model）。您嵌入的字符串并不总是用户的原始文本。任务说明、前缀、提示和文档格式可以是 模型（Model） 合同的一部分。

研究锚点：[14]、[18]理解嵌入向量（Embeddings）

24 - QUERY/DOCUMENT ASYMMETRY AND INSTRUCTIONS

The two sides do not always mean the same thing

A retrieval query and a document play different semantic roles. Many embedding models make that asymmetry explicit.

E5 uses one shared encoder but prepends different textual identifiers: “query:” for queries and “passage:” for passages. The paper reports that breaking the symmetry this way is important for some retrieval tasks. [14]

Newer instruction-aware embedding models generalize the idea. Qwen3 Embedding concatenates a task instruction with the query while leaving the document unchanged, allowing relevance to be conditioned on the requested task. [18]

This means a production pipeline must preserve the model’s input protocol. Encoding a corpus with the query template, omitting a required prefix, or changing an instruction after indexing can create a mismatch even though every vector has the expected dimension.

For symmetric tasks such as generic sentence similarity, the same textual role may be used on both sides. For asymmetric retrieval, query/document conventions can be part of the learned geometry.

Mental model. The string you embed is not always just the user’s raw text. Task instructions, prefixes, prompts, and document formatting can be part of the model contract.

Research anchors: [14], [18]

25 - 嵌入（Embedding） 尺寸

对应原文第 27 页；点击查看相邻原文对照页

25 - 嵌入（Embedding） DIMENSION 容量、存储、带宽和搜索成本 维度的 嵌入（Embedding） 存储 d 个数字。保持数据类型固定，向量（Vector） 有效负载随时间线性增长。

对于存储为 float32 的一百万个 向量（Vectors）：

而 256 维则需要大约 0.95 GiB。

这意味着原始 向量（Vector） 有效负载减少了 4 倍。真实索引添加元数据、图形结构、量化（Quantization） 状态、对齐和其他开销。

更高的维度可以赋予 模型（Model） 更强的表征能力，但“维度越多越好”是错误的。 检索（Retrieval） 质量、训练（Training）、数据、架构和 索引（Index） 行为都与维度相互作用。

危险的捷径是在 训练（Training） 之后截断任意 嵌入（Embedding） 向量（Vector） 并假设其第一个坐标是良好的低维 表示（Representation）。标准 训练（Training） 不保证信息按这种方式排序。

一些现代的 模型（Models） 经过明确的灵活维度训练。 Qwen3 嵌入（Embedding） 报告了 Matryoshka 支持，其官方 模型（Model） 卡公开了 模型（Model） 特定范围内的用户可选输出尺寸。 [18]

重要。仅当 模型（Model） 或经过验证的后处理方法支持时，降维才是安全的。匹配数组形状不是保留语义的 证据（Evidence）。

研究锚点：[15]、[18]理解嵌入向量（Embeddings）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

25 - EMBEDDING DIMENSION

Capacity, storage, bandwidth, and search cost

An embedding of dimension d stores d numbers. Holding datatype fixed, vector payload grows linearly with d.

For one million vectors stored as float32:

$$1{,}000{,}000 \times 1024 \times 4 \text{ bytes} \approx 3.81 \text{ GiB},$$

while 256 dimensions require approximately

$$0.95 \text{ GiB}.$$

That is a 4× reduction in raw vector payload. Real indexes add metadata, graph structures, quantization state, alignment, and other overhead.

Higher dimension can give a model more representational capacity, but "more dimensions is always better" is false. Retrieval quality, training, data, architecture, and index behavior all interact with dimension.

The dangerous shortcut is to truncate an arbitrary embedding vector after training and assume its first coordinates are a good lower-dimensional representation. Standard training does not guarantee that information is ordered that way.

Some modern models are explicitly trained for flexible dimension. Qwen3 Embedding reports Matryoshka support, and its official model cards expose user-selectable output dimensions within model-specific ranges. [18]

Important. Dimension reduction is safe only when the model or a validated post-processing method supports it. Matching array shape is not evidence that semantics were preserved.

Research anchors: [15], [18]

26 - 俄罗斯套娃代表

对应原文第 28 页；点击查看相邻原文对照页

26 - 俄罗斯套娃表示 训练有用的前缀，而不是盲目截断俄罗斯套娃 表示（Representation） 学习 (MRL) 训练高维 向量（Vector），以便选定的前缀本身就是有用的表示。如果

然后对于选择的尺寸 m∈M，前缀

针对该任务进行了明确的优化。 [15]

原始 MRL 论文使用嵌套维度，通常以大致对数间隔，因此一个 模型（Model） 可以支持多个 准确性（Accuracy）/效率操作点，而无需重新计算完全不同的 表示（Representation）。

64 128 256 full d 这与 PCA 或在 训练（Training） 之后取坐标的随机子集有根本的不同。

MRL 更改了 训练（Training） 目标，因此有用的信息被打包到嵌套前缀中。

如果每个前缀与 余弦相似度（Cosine Similarity） 一起使用，请在评分之前独立标准化该前缀，而不是重复使用完整 向量（Vector） 的范数。 [15]

理解框架（Mental Model）。灵活尺寸是学习属性，而不是 稠密向量（Dense Vectors） 的通用属性。

研究锚点：[15]、[18]理解嵌入向量（Embeddings）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

26 - MATRYOSHKA REPRESENTATIONS

Train useful prefixes instead of truncating blindly

Matryoshka Representation Learning (MRL) trains a high-dimensional vector so that selected prefixes are themselves useful representations. If

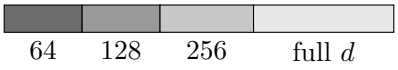
$$z \in \mathbb{R}^d,$$

then for chosen sizes $m \in \mathcal{M}$, the prefix

$$z_{1:m} \in \mathbb{R}^m$$

is explicitly optimized for the task. [15]

The original MRL paper uses nested dimensions, often spaced roughly logarithmically, so one model can support several accuracy/efficiency operating points without recomputing a completely different representation.



This is fundamentally different from PCA or taking a random subset of coordinates after training. MRL changes the training objective so useful information is packed into nested prefixes.

If each prefix is used with cosine similarity, normalize that prefix independently before scoring rather than reusing the norm of the full vector. [15]

Mental model. Flexible dimensions are a learned property, not a generic property of dense vectors.

Research anchors: [15], [18]

27 - 各向异性（Anisotropy） 和 表示（Representation） 几何形状

对应原文第 29 页；点击查看相邻原文对照页

27 - 各向异性（Anisotropy） 和表示（Representation） 几何 向量（Vector） 空间可能会塌陷成狭窄区域有用的嵌入空间（Embedding Space） 不能保证均匀地填充Rd。上下文和句子表示可能变得各向异性：许多 向量（Vectors） 占据一个狭窄的圆锥体或由一小组方向主导。 [11,21]

这很重要，因为相似性就会有一个压缩的 动态范围（Dynamic Range）。如果几乎每一对都有较高的 余弦相似度（Cosine Similarity），那么将真正相关的示例与不相关的示例区分开来就变得更加困难。

SimCSE 将 对比学习（Contrastive Learning） 与 表示空间（Representation Space） 的一致性得到改善。其分析认为，对比目标可以使奇异 值（Value） 谱变平，并使句子表示更加各向同性，同时保持正对的对齐。 [11]

各向异性（Anisotropy） 不是一种具有通用修复方法的通用标量缺陷。后处理方法、归一化（Normalization）、模型（Model） 架构、预训练（Pretraining） 物镜和 嵌入（Embedding） 微调（Fine-tuning） 都可以更改几何形状。

两个有用的概念是：

- 对齐：正值应该接近；

- 均匀性：不相关的样本不应全部塌陷到同一区域。

重要：高维 向量（Vector） 的有效几何多样性仍然较低。仅凭维度计数并不能告诉您空间是如何使用的。

研究锚点：[11]、[21] 理解 嵌入向量（Embeddings）

27 - ANISOTROPY AND REPRESENTATION GEOMETRY

A vector space can collapse into a narrow region

A useful embedding space is not guaranteed to fill $\mathbb{R}^d$ evenly. Contextual and sentence representations can become anisotropic: many vectors occupy a narrow cone or are dominated by a small set of directions. [11,21]

That matters because similarity then has a compressed dynamic range. If almost every pair has a high cosine similarity, separating truly related examples from unrelated ones becomes harder.

SimCSE connects contrastive learning with improved uniformity of the representation space. Its analysis argues that the contrastive objective can flatten the singular-value spectrum and make sentence representations more isotropic while preserving alignment of positive pairs. [11]

Anisotropy is not a universal scalar defect with one universal fix. Post-processing methods, normalization, model architecture, pretraining objectives, and embedding fine-tuning can all change the geometry.

Two useful concepts are:

- **alignment**: positives should be close;
- **uniformity**: unrelated samples should not all collapse into the same region.

Important. A high-dimensional vector can still have low effective geometric diversity. Dimension count alone does not tell you how the space is used.

Research anchors: [11], [21]

28 - 单 向量（Vector）、多 向量（Vector） 和稀疏表示

对应原文第 30 页；点击查看相邻原文对照页

28 - 单 向量（Vector）、多 向量（Vector） 和稀疏表示“嵌入（Embedding）”并不总是意味着一个 稠密向量（Dense Vector） 占主导地位的 理解框架（Mental Model） 是每个文本一个 稠密向量（Dense Vector），但 检索（Retrieval） 表示更广泛。

Single-向量（Vector）密集。一个句子或文档被压缩成一个向量（Vector）。这是经典的 SBERT/DPR/E5 风格的界面，很容易使用 索引（Index）。 [9,10,14]

多个 向量（Vector）/延迟交互。文档可以保留多个 词元（Token） 或区域级别 向量（Vectors） 并推迟一些交互直到评分。这可以以更高的存储和评分成本保留更细粒度的信息。

稀疏学习表示。 模型（Model） 可以生成类似稀疏加权 词表（Vocabulary） 的 向量（Vector），而不是密集的低维 向量（Vector）。

BGE-M3 是一个有用的现代示例，因为一个 模型（Model） 支持密集 检索（Retrieval）、稀疏 检索（Retrieval） 和多 向量（Vector） 检索（Retrieval）。 [17]

这些表示解决了不同的压缩/交互权衡。单个稠密向量（Dense Vector）最大程度地方便了预计算，但它必须将所有相关信息压缩为一个点。多 向量（Vector） 方法保留更多局部细节。稀疏表示以学习的形式保留词汇维度。

重要：在讨论现代 检索（Retrieval） 系统时，请勿使用“嵌入（Embedding） 模型（Model）”作为“单个 稠密向量（Dense Vector） 模型（Model）”的同义词。

研究锚点：[9]、[10]、[14]、[17] 了解 嵌入向量（Embeddings） 说明性 表示（Representation） 大小 1,024 维单 向量（Vector） 表示（Representation） 每个文档存储 1,024 个标量 值（Values）。

保留维度 128 的 128 词元（Token） 向量（Vectors） 在压缩前存储 16,384 值（Values） - 16 倍。真实的多 向量（Vector） 系统可以量化、修剪或以其他方式压缩这些表示，因此这是 表示（Representation） 大小的示例，而不是通用存储比率。

28 - SINGLE-VECTOR, MULTI-VECTOR, AND SPARSE REPRESENTATIONS

“Embedding” does not always mean one dense vector

The dominant mental model is one dense vector per text, but retrieval representations are broader.

Single-vector dense. A sentence or document is compressed into one vector. This is the classic SBERT/DPR/E5-style interface and is easy to index. [9,10,14]

Multi-vector / late interaction. A document can retain multiple token- or region-level vectors and defer some interaction until scoring. This preserves finer-grained information at greater storage and scoring cost.

Sparse learned representations. A model can produce a sparse weighted vocabulary-like vector rather than a dense low-dimensional vector.

BGE-M3 is a useful modern example because one model supports dense retrieval, sparse retrieval, and multi-vector retrieval. [17]

These representations solve different compression/interaction trade-offs. A single dense vector is maximally convenient for precomputation, but it must compress all relevant information into one point. Multi-vector approaches keep more local detail. Sparse representations preserve lexical dimensions in a learned form.

Important. Do not use “embedding model” as a synonym for “single dense vector model” when discussing modern retrieval systems.

Illustrative representation size

A 1,024-dimensional single-vector representation stores 1,024 scalar values per document. Retaining 128 token vectors of dimension 128 stores 16,384 values before compression - 16x as many. Real multi-vector systems may quantize, prune, or otherwise compress these representations, so this is a representation-size example, not a universal storage ratio.

Research anchors: [9], [10], [14], [17]

29 - 多语言和多模式 嵌入向量（Embeddings）

对应原文第 31 页；点击查看相邻原文对照页

29 - 多语言和多模式 嵌入向量（Embeddings） 共享空间可以跨语言和模式 同样的 表示（Representation） 学习理念可以对齐不是用同一种语言编写的对象，或者根本不是文本。

多语言 嵌入（Embedding） 模型（Model） 可以训练来自不同语言的语义对应文本以占据一个空间的兼容区域。现代 模型（Models）（例如 BGE-M3 和 Qwen3 嵌入（Embedding））报告多语言 训练（Training） 和跨多种语言的评估。 [17,18]

多模式 模型（Model） 可以对齐不同的数据类型。 CLIP 在图像-文本对上联合训练图像 编码器（Encoder） 和文本 编码器（Encoder），以便匹配的图像和描述接收兼容的表示。这使得文本到图像和图像到文本 检索（Retrieval） 成为可能，而无需通过相同的原始输入 编码器（Encoder） 强制两种模式。 [12]

核心（Core） 模式仍然是 fa(xa), fb(xb)→ 共享比较空间，但每个模态可以有自己的 编码器（Encoder） 和预处理。

共享空间的存在并不意味着 向量（Vectors） 对于每个任务都是可以互换的。

训练（Training） 对和目标决定了强调哪些跨模式或跨语言关系。

理解框架（Mental Model）。 嵌入（Embedding） 是 表示（Representation） 接口。如果 训练（Training） 将两个域对齐为兼容的几何形状，则相似性可以弥合语言、模式或数据类型。

研究锚点：[12]、[17]、[18] 理解 嵌入向量（Embeddings）

29 - MULTILINGUAL AND MULTIMODAL EMBEDDINGS

Shared spaces can cross languages and modalities

The same representation-learning idea can align objects that are not written in the same language, or not text at all.

A multilingual embedding model can train semantically corresponding texts from different languages to occupy compatible regions of one space. Modern models such as BGE-M3 and Qwen3 Embedding report multilingual training and evaluation across many languages. [17,18]

A multimodal model can align different data types. CLIP jointly trains an image encoder and a text encoder on image-text pairs so that matched images and descriptions receive compatible representations. This makes text-to-image and image-to-text retrieval possible without forcing both modalities through the same raw input encoder. [12]

The core pattern is still

$$f_a(x_a),\ f_b(x_b) \rightarrow \text{shared comparison space},$$

but each modality can have its own encoder and preprocessing.

The existence of a shared space does not mean the vectors are interchangeable for every task. The training pairs and objective determine what cross-modal or cross-lingual relationships are emphasized.

Mental model. Embedding is a representation interface. If training aligns two domains into a compatible geometry, similarity can bridge languages, modalities, or data types.

Research anchors: [12], [17], [18]

30 - 评估 嵌入向量（Embeddings）

对应原文第 32 页；点击查看相邻原文对照页

30 - EV ALUATING 嵌入向量（Embeddings） 一个 基准测试（Benchmark） 分数无法定义通用的 嵌入（Embedding） 嵌入（Embedding） 模型（Models） 用于不同的任务：语义文本相似性、检索（Retrieval）、分类、聚类、对分类、重排序（Reranking） 支持、双文本挖掘等。

创建 MTEB 的部分原因是仅评估一项任务无法获得完整的图像。其最初的 基准测试（Benchmark） 涵盖了八个任务类别，并发现没有单一方法主导该评估中的每项任务。 [16]

不同的任务需要不同的指标：

- STS 通常使用预测相似性和人类评分之间的相关性；

- 检索（Retrieval） 使用排名指标，例如 召回率（Recall）、MRR、nDCG 或特定于任务的变体；

- 分类评估基于嵌入向量（Embeddings） 构建的分类器或探针；

- 聚类衡量几何结构是否支持有意义的分组。

即使 检索（Retrieval） 模型（Model） 的原始余弦分数作为概率校准得不好，它也可能非常出色。

很好地聚类主题的 模型（Model） 可能不是最好的非对称 查询（Query） 文档 检索器（Retriever）。

基准还取决于语言、领域、文本长度、说明、数据污染和评估实施。当前的 MTEB 项目已经远远超出了原始文本 基准测试（Benchmark） 的范围，包括更广泛的多语言和多模式评估，因此排行榜声明应始终在日期和 基准测试（Benchmark） 范围内。 [8]

重要。使用您真正关心的任务和数据分布而不是全局平均分数来选择 嵌入（Embedding） 模型（Model）。

研究锚点：[8]、[16] 理解 嵌入向量（Embeddings）

30 - EVALUATING EMBEDDINGS

One benchmark score cannot define a universal embedding

Embedding models are used for different tasks: semantic textual similarity, retrieval, classification, clustering, pair classification, reranking support, bitext mining, and more.

MTEB was created partly because evaluating only one task gives an incomplete picture. Its original benchmark covered eight task categories and found that no single method dominated every task in that evaluation. [16]

Different tasks require different metrics:

- STS often uses correlation between predicted similarity and human scores;
- retrieval uses ranking metrics such as Recall, MRR, nDCG, or task-specific variants;
- classification evaluates a classifier or probe built on embeddings;
- clustering measures whether the geometry supports meaningful grouping.

A retrieval model can be excellent even if its raw cosine scores are poorly calibrated as probabilities. A model that clusters topics well might not be the best asymmetric query-document retriever.

Benchmarks also depend on language, domain, text length, instructions, data contamination, and evaluation implementation. The current MTEB project has expanded far beyond the original text benchmark, including broader multilingual and multimodal evaluation, so leaderboard claims should always be date- and benchmark-scoped. [8]

Important. Choose an embedding model using the task and data distribution you actually care about, not one global average score.

Research anchors: [8], [16]

31 - 嵌入向量（Embeddings） 穿着破布

对应原文第 33 页；点击查看相邻原文对照页

31 - 嵌入向量（Embeddings） IN RAG 嵌入向量（Embeddings） 选择候选人；它们不会生成答案 在典型的密集 RAG 流水线（Pipeline） 中：

1. 拆分或以其他方式定义可检索单位；

2. 将这些单元编码为文档嵌入（Document Embeddings）；

3.构建一个向量（Vector） 索引（Index）；

4. 使用正确的查询（Query）协议对用户的查询（Query）进行编码；

5.检索高分候选人；

6. 可选择对它们重新排序；

7. 将选定的文本发送至生成器（Generator）。

因此，嵌入（Embedding） 模型（Model） 会影响哪个 证据（Evidence） 达到 LLM，但它不是 生成器（Generator），并且它不包含每个 查询（Query） 向量（Vector） 内的完整外部语料库。

文档嵌入（Document Embedding） 是针对相似性目标进行优化的有损 表示（Representation）。它对于候选选择非常有用，因为比较 向量（Vectors） 比在每个文档上联合运行强大的交叉 编码器（Encoder） 或生成 模型（Model） 更便宜。

这会创建几种单独的故障模式：错误的块边界、不理解域的 嵌入（Embedding） 模型（Model）、查询（Query）/文档格式不匹配、近似搜索未命中或 重排序器（Reranker）/生成器（Generator） 失败。

重要： RAG 不是“在 嵌入向量（Embeddings） 中存储知识”。源文本仍然是 证据（Evidence）。

嵌入向量（Embeddings） 是用于查找候选文本的 检索（Retrieval） 表示（Representation）。

研究锚点：[10]、[14]、[18] 理解 嵌入向量（Embeddings）

31 - EMBEDDINGS IN RAG

Embeddings choose candidates; they do not generate the answer

In a typical dense RAG pipeline:

1. split or otherwise define retrievable units;
2. encode those units into document embeddings;
3. build a vector index;
4. encode the user's query using the correct query protocol;
5. retrieve high-scoring candidates;
6. optionally rerank them;
7. send selected text to the generator.

The embedding model therefore affects which evidence reaches the LLM, but it is not the generator and it does not contain the full external corpus inside each query vector.

A document embedding is a lossy representation optimized for a similarity objective. It is useful for candidate selection because comparing vectors is cheaper than running a powerful cross-encoder or generative model jointly over every document.

This creates several separate failure modes: bad chunk boundaries, an embedding model that does not understand the domain, a query/document formatting mismatch, an approximate-search miss, or a reranker/generator failure.

Important. RAG is not "store knowledge in embeddings." The source text remains the evidence. Embeddings are a retrieval representation used to find candidate text.

Research anchors: [10], [14], [18]

32 - 生产耦合和版本控制

对应原文第 34 页；点击查看相邻原文对照页

32 - 生产耦合和版本 向量（Vectors） 仅在兼容的 嵌入空间（Embedding Space） 内进行比较 存储的 向量（Vector） 默默地取决于许多选择：

- 模型（Model） 重量和准确版本；

- 分词器（Tokenizer） 和文本预处理；

- 查询（Query）/文档提示或说明；

- 池化（Pooling）/读出规则；

- 嵌入（Embedding） 尺寸或截断设置；

- 归一化（Normalization）；

- 输出dtype 或量化（Quantization）；

- 相似性度量。

更改其中任何一个都可能导致新的 向量（Vectors） 与现有的 索引（Index） 不兼容。两个 模型（Models） 都可以输出 1024 个维度，但位于不相关的坐标系中。

这就是为什么 嵌入（Embedding）-模型（Model） 升级经常需要重新 嵌入（Embedding） 语料库。尺寸匹配是不够的。即使来自同一系列的新检查点也可以旋转、重新缩放或以其他方式重塑空间。

因此，生产系统应将 嵌入（Embedding） 合约与 索引（Index） 一起版本化。有用的元数据包括 模型（Model） 标识符、修订版、预处理模板、角色/指令、输出维度、归一化（Normalization） 约定和指标。

当前的 嵌入（Embedding） 库将其中许多公开为配置而不是通用默认值。

例如Qwen3，支持用户自定义任务指令，维度灵活； Sentence Transformers 公开了 池化（Pooling） 和相似性模块。 [18,20]

理解框架（Mental Model）。 嵌入（Embedding） 索引（Index） 与生成它的 编码器（Encoder） 耦合。将 编码器（Encoder） 更改视为架构更改，而不是装饰性的 模型（Model） 交换。

研究锚点：[18]、[20]理解嵌入向量（Embeddings）

32 - PRODUCTION COUPLING AND VERSIONING

Vectors are only comparable inside a compatible embedding space

A stored vector silently depends on many choices:

- model weights and exact version;
- tokenizer and text preprocessing;
- query/document prompts or instructions;
- pooling/readout rule;
- embedding dimension or truncation setting;
- normalization;
- output dtype or quantization;
- similarity metric.

Changing any of these can make new vectors incompatible with an existing index. Two models can both output 1024 dimensions yet inhabit unrelated coordinate systems.

This is why embedding-model upgrades often require re-embedding the corpus. A dimension match is not enough. Even a new checkpoint from the same family can rotate, rescale, or otherwise reshape the space.

Production systems should therefore version the embedding contract alongside the index. Useful metadata includes model identifier, revision, preprocessing template, role/instruction, output dimension, normalization convention, and metric.

Current embedding libraries expose many of these as configuration rather than universal defaults. Qwen3, for example, supports user-defined task instructions and flexible dimensions; Sentence Transformers exposes pooling and similarity modules. [18,20]

Mental model. An embedding index is coupled to the encoder that produced it. Treat encoder changes like schema changes, not cosmetic model swaps.

Research anchors: [18], [20]

33 - 透明的 Pytorch 实现

对应原文第 35 页；点击查看相邻原文对照页

33 - 透明的 PYTORCH 实现查找、屏蔽 平均池化（Mean Pooling） 和 归一化（Normalization） 以下代码显示了三个不同的阶段：词元（Token） 查找、池化（Pooling） 上下文状态和可选的 L2 归一化（Normalization）。

代码注释：1）词元嵌入（Token Embedding）查找

嵌入（Embedding） = torch.nn.嵌入（Embedding）(V, d)

x = 嵌入（Embedding）(ids) # [1, 3, 64]

代码注释：2）想象“hidden”是上下文 Transformer 模型（Transformer） 输出

hide = x # 教学占位符 掩码（Mask） = torch.张量（Tensor）([[1, 1, 1]], dtype=torch.bool)

合并=（隐藏*权重）。总和(dim=1) / 权重。总和（暗淡=1）

代码注释：[1, 64]

代码注释：3）如果 模型（Model） 合约需要的话，则进行单位标准化

z = F.normalize(pooled, p=2, dim=-1) 在真实文本中-嵌入（Embedding） 模型（Model），隐藏将由上下文编码器（Encoder） 生成，并且 池化（Pooling）/读数必须与 模型（Model） 的 训练（Training） 配置匹配。有些 模型（Models） 使用 平均池化（Mean Pooling），有些是特殊的 词元（Token），有些是最后/EOS 状态，有些是附加投影。

重要：该代码是教学性的。不要仅仅因为 张量（Tensor） 形状有效，就用此通用平均值 池化（Pooling） 配方替换 模型（Model） 记录的 池化（Pooling） 或提示协议。

研究锚点：[9]、[18]、[19]、[20] 理解 嵌入向量（Embeddings）

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

33 - A TRANSPARENT PYTORCH IMPLEMENTATION

Lookup, masked mean pooling, and normalization

The following code shows three distinct stages: token lookup, pooling contextual states, and optional L2 normalization.

```
import torch
import torch.nn.functional as F

# 1) Token embedding lookup
V, d = 1000, 64
embedding = torch.nn.Embedding(V, d)
ids = torch.tensor([[4, 1, 7]]) # [B=1, T=3]
x = embedding(ids) # [1, 3, 64]

# 2) Imagine ‘hidden‘ is the contextual Transformer output
hidden = x # pedagogical placeholder
mask = torch.tensor([[1, 1, 1]], dtype=torch.bool)

weights = mask.unsqueeze(-1).to(hidden.dtype)
pooled = (hidden * weights).sum(dim=1) / weights.sum(dim=1)
                                        # [1, 64]

# 3) Unit-normalize if the model contract expects it
z = F.normalize(pooled, p=2, dim=-1)
```

In a real text-embedding model, `hidden` would be produced by the contextual encoder, and the pooling/readout must match the model's training configuration. Some models use mean pooling, some a special token, some the last/EOS state, and some an additional projection.

Important. This code is pedagogical. Do not replace a model's documented pooling or prompt protocol with this generic mean-pooling recipe merely because the tensor shapes work.

Research anchors: [9], [18], [19], [20]

34 - 常见的实施错误

对应原文第 36 页；点击查看相邻原文对照页

34 - 常见实施错误 大多数失败是合约不匹配，而不是 向量（Vector） 算术错误

1. 池化（Pooling） 填充 词元（Tokens）.平均池化（Mean Pooling） 通常应遵循 注意力机制（Attention） 掩码（Mask）。

2. 混合标准化和非标准化 向量（Vectors）。这会默默地改变分数语义。

3. 使用错误的相似性度量。当规范不同时，Cosine、点积（Dot Product） 和 L2 的排名可能不同。

4.省略查询（Query）/文档前缀或指令。一些模型（Models）经过非对称输入格式的显式训练。

5. 在没有 MRL 支持的情况下更改尺寸。任意截断不需要保留有用的几何形状。

6. 比较向量（Vectors）与不同模型（Model）版本。相同的形状并不意味着相同的坐标系。

7. 更改预处理后重新使用语料库索引（Index）。分词（Tokenization）、模板和截断都可以更改向量（Vectors）。

8. 将原始 LM 隐藏状态视为一个插入项 检索（Retrieval） 模型（Model）。良好的语言 - 模型（Model）表示不会自动针对最近邻居语义进行优化。

9. 忘记长输入可能会被截断。生成的 向量（Vector） 可能仅表示保留的前缀或 模型（Model） 特定的窗口行为。

理解框架（Mental Model）。写下 嵌入（Embedding） 合约。 模型（Model）、输入格式、读数、维度、归一化（Normalization） 和度量应该是显式配置，而不是部落知识。

研究锚点：[9]、[14]、[15]、[18]、[20] 理解 嵌入向量（Embeddings）

34 - COMMON IMPLEMENTATION MISTAKES

Most failures are contract mismatches, not vector arithmetic bugs

1. Pooling padding tokens. Mean pooling should generally respect the attention mask.

2. Mixing normalized and unnormalized vectors. This silently changes score semantics.

3. Using the wrong similarity metric. Cosine, dot product, and L2 can rank differently when norms vary.

4. Omitting query/document prefixes or instructions. Some models were explicitly trained with asymmetric input formatting.

5. Changing dimensions without MRL support. Arbitrary truncation need not preserve useful geometry.

6. Comparing vectors from different model versions. Same shape does not imply same coordinate system.

7. Reusing a corpus index after changing preprocessing. Tokenization, templates, and truncation can all change vectors.

8. Treating raw LM hidden states as a drop-in retrieval model. Good language-model representations are not automatically optimized for nearest-neighbor semantics.

9. Forgetting that long inputs may be truncated. The resulting vector may represent only the retained prefix or model-specific window behavior.

Mental model. Write the embedding contract down. Model, input format, readout, dimension, normalization, and metric should be explicit configuration, not tribal knowledge.

Research anchors: [9], [14], [15], [18], [20]

35 - 常见的误解

对应原文第 37 页；点击查看相邻原文对照页

35 - 常见误解 创建错误 理解框架（Mental Model） 的快捷方式 误解更正“词元（Token） ID 是 嵌入（Embedding）。” ID 选择一行。 向量（Vector） 行是 嵌入（Embedding）。

“每个 嵌入（Embedding） 维度都有一个含义。”表示通常分布在维度和方向上。

“同一个词总是有相同的 嵌入（Embedding）。”它的输入行可能是固定的，但上下文隐藏状态取决于上下文。

“BERT/LLM 隐藏状态自动是好的 检索（Retrieval） 嵌入向量（Embeddings）。”相似几何通常需要合适的读数和 训练（Training） 物镜。

“余弦和 点积（Dot Product） 是相同的。”他们同意单位归一化的 向量（Vectors），而不是任意的 向量（Vectors）。

“更多维度总是会改进 检索（Retrieval）。” Dimension 将容量与存储/计算进行交换，并取决于 训练（Training）。

“如果尺寸匹配，您可以将 嵌入向量（Embeddings） 与不同的 模型（Models） 进行比较。”不同的编码器定义不同的坐标系。

“RAG 将事实存储在 向量（Vectors） 中。” 向量（Vectors） 帮助检索源文本；它们不是源文本本身。

重要：“嵌入（Embedding）”一词描述的是 表示（Representation），而不是一种通用架构、目标、池化（Pooling） 规则或相似性度量。

研究锚点：[4]、[9]、[10]、[15]、[20] 理解 嵌入向量（Embeddings）

35 - COMMON MISCONCEPTIONS

Shortcuts that create the wrong mental model

Misconception	Correction
“A token ID is an embedding.”	An ID selects a row. The vector row is the embedding.
“Each embedding dimension has one meaning.”	Representations are generally distributed across dimensions and directions.
“The same word always has the same embedding.”	Its input row may be fixed, but contextual hidden states depend on context.
“BERT/LLM hidden states are automatically good retrieval embeddings.”	Similarity geometry usually needs a suitable readout and training objective.
“Cosine and dot product are the same.”	They agree on unit-normalized vectors, not arbitrary vectors.
“More dimensions always improve retrieval.”	Dimension trades capacity against storage/compute and depends on training.
“You can compare embeddings from different models if dimensions match.”	Different encoders define different coordinate systems.
“RAG stores facts in vectors.”	Vectors help retrieve source text; they are not the source text itself.

Important. The word “embedding” describes a representation, not one universal architecture, objective, pooling rule, or similarity metric.

Research anchors: [4], [9], [10], [15], [20]

36 - 完整的 理解框架（Mental Model）

对应原文第 38 页；点击查看相邻原文对照页

36 - 一条链中的完整 理解框架（Mental Model） 嵌入向量（Embeddings） 嵌入向量（Embeddings） 作为九个想法的序列最容易记住。

1. 离散化。 分词（Tokenization） 将文本映射到 词元（Token） ID。

2. 向上看。 ID 从 词表（Vocabulary） 嵌入矩阵（Embedding Matrix） 中选择学习行。

3.结合情境。 Transformer 模型（Transformer） 层将基本行转换为上下文相关的隐藏状态。

4. 读出。需要时，池化（Pooling） 或 模型（Model） 特定规则将可变长度状态转换为固定大小文本 向量（Vector）。

5. 形状几何。对比或特定任务 训练（Training） 使有用的配对得分比阴性配对更好。

6. 选择一个度量。点积（Dot Product）、余弦和 L2 仅在使用 模型（Model） 的 归一化（Normalization） 约定时才有意义。

7. 选择容量。嵌入（Embedding） 尺寸控制向量（Vector） 负载，只有训练（Training） 支持时才能灵活。

8.索引（Index）。 检索（Retrieval）系统预先计算文档向量（Vectors）并对其进行高效搜索； 索引（Index） 与 编码器（Encoder） 是分开的。

9. 对合同进行版本控制。模型（Model）、预处理、提示、池化（Pooling）、维度、归一化（Normalization） 和指标必须保持兼容。

静态词 向量（Vectors）、上下文 词元（Token） 表示、句子嵌入（Sentence Embeddings）、密集检索器、俄罗斯套娃 向量（Vectors）、多语言空间和多模态空间都是同一个广泛思想的变体：学习 表示（Representation），其中下游结构变得更容易计算。

理解框架（Mental Model）。 嵌入（Embedding） 是有目的而学习的有用几何图形。要了解其中之一，请询问映射的内容、向量（Vector） 是如何生成的、什么目标塑造了空间以及如何比较 向量（Vectors）。

研究锚点：[1]–[20] 理解 嵌入向量（Embeddings）

36 - THE COMPLETE MENTAL MODEL

Embeddings in one chain

Embeddings are easiest to remember as a sequence of nine ideas.

1. **Discretize.** Tokenization maps text to token IDs.
2. **Look up.** IDs select learned rows from a vocabulary embedding matrix.
3. **Contextualize.** Transformer layers turn base rows into context-dependent hidden states.
4. **Read out.** A pooling or model-specific rule converts variable-length states into a fixed-size text vector when needed.
5. **Shape geometry.** Contrastive or task-specific training makes useful pairs score better than negatives.
6. **Choose a metric.** Dot product, cosine, and L2 only become meaningful with the model's normalization convention.
7. **Choose capacity.** Embedding dimension controls vector payload and can be flexible only when training supports it.
8. **Index.** Retrieval systems precompute document vectors and search them efficiently; the index is separate from the encoder.
9. **Version the contract.** Model, preprocessing, prompts, pooling, dimension, normalization, and metric must remain compatible.

Static word vectors, contextual token representations, sentence embeddings, dense retrievers, Matryoshka vectors, multilingual spaces, and multimodal spaces are all variations on the same broad idea: learn a representation in which downstream structure becomes easier to compute.

Mental model. An embedding is useful geometry learned for a purpose. To understand one, ask what was mapped, how the vector was produced, what objective shaped the space, and how vectors are compared.

Research anchors: [1]–[20]

参考

对应原文第 39 页；点击查看相邻原文对照页

参考文献 主要论文和当前实施来源

1. Y. Bengio、R. Ducharme、P. Vincent、C. Jauvin。神经概率语言 模型（Model）。 JMLR 3，2003 年。

2. T. Mikolov、K. Chen、G. Corrado、J. Dean。向量（Vector） 空间中单词表示的高效估计。

arXiv：1301.3781，2013。

3. J. Pennington、R. Socher、C. Manning.GloVe：Word 表示（Representation） 的全局 向量（Vectors）。 EMNLP 2014。

4. A. Vaswani 等人。注意力机制（Attention） 就是您所需要的。 NeurIPS 2017。arXiv：1706.03762。

5. O. Press、L. Wolf。使用输出 嵌入（Embedding） 改进语言 模型（Models）。 EACL 2017。arXiv：1608.05859。

6. M. Peters 等人的《深度语境化单词表示》。 NAACL 2018。arXiv：1802.05365。

7. J. Devlin，M.-W。 Chang、K. Lee、K. Toutanova.BERT：用于语言理解的深度双向变压器的 Pre-训练（Training）。 NAACL 2019。arXiv：1810.04805。

8. MTEB 项目文档和存储库。当前 基准测试（Benchmark） 范围、任务和评估工具。

2026 年 9 月咨询。

9. N. Reimers, I. Gurevych.Sentence-BERT: 句子嵌入（Sentence Embeddings） 使用 Siamese BERT 网络。 EMNLP

2019.arXiv:1908.10084。

10. V. Karpukhin 等人。用于开放域问答的密集段落 检索（Retrieval）。 EMNLP 2020。

arXiv：2004.04906。

11. T. 高，X. 姚，D. Chen.SimCSE：句子嵌入（Sentence Embeddings） 的简单 对比学习（Contrastive Learning）。 EMNLP 2021。

arXiv：2104.08821。

12. A. Radford 等人。从自然语言监督中学习可转移视觉 模型（Models）。 ICML 2021。

arXiv：2103.00020。

13. A. Neelakantan 等人。对比 Pre-训练（Training） 的文本和代码 嵌入向量（Embeddings）。 arXiv：2201.10005，2022。

14. L. Wang 等人通过弱监督对比 Pre-训练（Training） 编写文本 嵌入向量（Embeddings）。 arXiv：2212.03533，2022。

15. A. Kusupati 等人。Matryoshka 表示（Representation） 学习。 NeurIPS 2022。arXiv：2205.13147。

16. N. Muennighoff 等人.MTEB：大量文本 嵌入（Embedding） 基准测试（Benchmark）。 EACL 2023。arXiv：2210.07316。

17. J. Chen 等人.BGE M3-嵌入（Embedding）：通过自我知识蒸馏实现多语言、多功能、多粒度文本 嵌入向量（Embeddings）。 arXiv：2402.03216，2024。

18. Y.Zhang 等人。Qwen3 嵌入（Embedding）：通过基金会 模型（Models） 推进文本 嵌入（Embedding） 和 重排序（Reranking）。

arXiv：2506.05176，2025。

19.PyTorch 文档。火炬.nn.嵌入（Embedding）。当前文档查阅于 2026 年 9 月。

20. 句子转换器文档。 池化（Pooling）、相似函数、归一化（Normalization） 和 嵌入（Embedding） 实用程序。

当前文档查阅于 2026 年 9 月。

21. B. Li 等人关于来自预训练语言 模型（Models） 的 句子嵌入（Sentence Embeddings）。 EMNLP 2020。arXiv：2011.05864。

了解 嵌入向量（Embeddings）

REFERENCES

Primary papers and current implementation sources

1. Y. Bengio, R. Ducharme, P. Vincent, C. Jauvin. *A Neural Probabilistic Language Model.* JMLR 3, 2003.
2. T. Mikolov, K. Chen, G. Corrado, J. Dean. *Efficient Estimation of Word Representations in Vector Space.* arXiv:1301.3781, 2013.
3. J. Pennington, R. Socher, C. Manning. *GloVe: Global Vectors for Word Representation.* EMNLP 2014.
4. A. Vaswani et al. *Attention Is All You Need.* NeurIPS 2017. arXiv:1706.03762.
5. O. Press, L. Wolf. *Using the Output Embedding to Improve Language Models.* EACL 2017. arXiv:1608.05859.
6. M. Peters et al. *Deep Contextualized Word Representations.* NAACL 2018. arXiv:1802.05365.
7. J. Devlin, M.-W. Chang, K. Lee, K. Toutanova. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.* NAACL 2019. arXiv:1810.04805.
8. MTEB project documentation and repository. Current benchmark scope, tasks, and evaluation tooling. Consulted September 2026.
9. N. Reimers, I. Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.* EMNLP 2019. arXiv:1908.10084.
10. V. Karpukhin et al. *Dense Passage Retrieval for Open-Domain Question Answering.* EMNLP 2020. arXiv:2004.04906.
11. T. Gao, X. Yao, D. Chen. *SimCSE: Simple Contrastive Learning of Sentence Embeddings.* EMNLP 2021. arXiv:2104.08821.
12. A. Radford et al. *Learning Transferable Visual Models From Natural Language Supervision.* ICML 2021. arXiv:2103.00020.
13. A. Neelakantan et al. *Text and Code Embeddings by Contrastive Pre-Training.* arXiv:2201.10005, 2022.
14. L. Wang et al. *Text Embeddings by Weakly-Supervised Contrastive Pre-training.* arXiv:2212.03533, 2022.
15. A. Kusupati et al. *Matryoshka Representation Learning.* NeurIPS 2022. arXiv:2205.13147.
16. N. Muennighoff et al. *MTEB: Massive Text Embedding Benchmark.* EACL 2023. arXiv:2210.07316.
17. J. Chen et al. *BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation.* arXiv:2402.03216, 2024.
18. Y. Zhang et al. *Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models.* arXiv:2506.05176, 2025.
19. PyTorch documentation. `torch.nn.Embedding`. Current documentation consulted September 2026.
20. Sentence Transformers documentation. Pooling, similarity functions, normalization, and embedding utilities. Current documentation consulted September 2026.
21. B. Li et al. *On the Sentence Embeddings from Pre-trained Language Models.* EMNLP 2020. arXiv:2011.05864.